摘 要

物联网和嵌入式操作系统在各个领域被广泛应用的同时，也面临着严峻的安全挑战。由于物联网和嵌入式操作系统常搭载在具有与硬件紧密集成特性的微控制器上，利用自动化的漏洞测试技术分析其固件的安全漏洞常常面临着棘手的硬件依赖问题。通过固件仿真技术来模拟嵌入式系统，使包含嵌入式系统的固件在商用计算机和服务器的软件模拟器上运行，是克服固件分析过程中的硬件依赖问题的有效方法。然而，当前的固件仿真技术仍存在很多问题，无法很好地解决嵌入式设备的各种外设难以模拟的困境，同时业界对基于固件仿真的完善且高效的物联网操作系统模糊测试工具的需求也越来越强烈。

本文研究了使用基于固件仿真的物联网模糊测试方法，改进了现有固件仿真技术，设计并实现了针对复杂物联网操作系统的模糊测试工具。工具利用库函数替换技术，对嵌入式操作系统的驱动库与内部库中负责与外设交互的关键函数进行替换，解决了固件仿真的外设依赖问题使得对来自同一硬件开发商甚至使用同一操作系统的不同固件具有很好的可重用性。

通过实验测试，本文的设计的工具对嵌入式操作系统的日志系统，网络与串口通信，文件系统等功能均能很好地模拟。使用该工具进行模糊测试的效果优于采用类似方案的其他模糊测试工具，具有很好的有效性和实用性，为物联网和嵌入式操作系统的安全保障提供了一种新的技术手段。

关键词：物联网安全 操作系统 模糊测试 固件仿真

ABSTRACT

While the Internet of Things (IoT) and embedded operating systems are widely used in various fields, they also face severe security challenges. Due to the fact that IoT and embedded operating systems are often installed on microcontrollers with tightly integrated hardware features, analyzing the security vulnerabilities of their firmware using automated vulnerability testing techniques often faces thorny hardware dependency issues. Simulating embedded systems using firmware simulation technology is an effective way to overcome hardware dependency issues in firmware analysis by running firmware containing embedded systems on software simulators on commercial computers and servers. However, current firmware simulation technology still has many problems and cannot solve the dilemma of various peripherals that are difficult to simulate in embedded devices well. At the same time, the industry's demand for perfect and efficient IoT operating system fuzz testing tools based on firmware simulation is becoming increasingly strong.

This thesis studies the use of firmware simulation-based IoT fuzz testing methods, improves existing firmware simulation technology, and designs and implements a fuzz testing tool for complex IoT operating systems. The tool uses library function replacement technology to replace key functions responsible for interacting with peripherals in the driver library and internal library of embedded operating systems, solving the problem of peripheral dependency in firmware simulation and making different firmware from the same hardware developer or even using the same operating system have good reusability.

Through experimental testing, this thesis’s designed tool can simulate the log system, network and serial communication, file system and other functions of embedded operating systems well. The effect of using this tool for fuzz testing is better than other similar fuzz testing tools that adopt similar schemes, which has good effectiveness and practicality and provides a new technical means for the security guarantee of IoT and

embedded operating systems.

Key words: **IoT Security** **Operating System** **Fuzz Testing** **Firmware Simulation**

目 录

摘 要 **1**
ABSTRACT **2**
目 录 ..... **i**
第一章 绪论 **1**
1.1 研究背景 1
1.2 研究现状 2
1.2.1 固件仿真技术研究现状 3
1.2.2 嵌入式系统模糊测试技术发展现状 6
1.3 研究内容与章节安排 8
第二章 背景知识 **9**
2.1 物联网操作系统背景知识 9
2.1.1 FreeRTOS 操作系统 10
2.1.2 LiteOS 操作系统 11
2.1.3 Zephyr 操作系统 12
2.2 基于 ARM 微控制器的物联网设备硬件基础知识 13
2.2.1 Arm Cortex M 架构特性 14
2.3 模拟仿真平台 15
2.3.1 QEMU 模拟器 15
2.3.2 Unicorn 引擎 16
2.4 模糊测试 17
2.4.1 模糊测试技术简介 17
2.4.2 嵌入式设备模糊测试发展现状 18
2.4.3 模糊测试工具 AFL 18
2.5 本章小结 21
第三章 设计与实现 **22**
3.1 整体结构设计 22
3.2 固件模拟与中断控制模块设计 23

3.3 库函数替换模块设计 26
3.4 FuzzerIO 模块 40
3.5 本章小结 42
第四章 测试 44
4.1 测试案例与测试环境 44
4.1.1 测试案例选择 44
4.1.2 测试环境 45
4.2 模拟仿真功能测试 45
4.2.1 测试方案 45
4.2.2 测试结果 46
4.3 模糊测试功能测试 51
4.3.1 测试方案 51
4.3.2 测试结果 52
4.4 本章小结 53
第五章 总结与展望 54
5.1 论文总结 54
5.2 未来工作展望 54
致 谢 56
参考文献 57

第一章 绪论

1.1 研究背景

近年来，随着物联网（IoT）技术迅速出现并不断发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据《中国移动互联网发展报告（2022）》，搭载了物联网传感器的可穿戴设备、智能家居、智慧医疗、车联网、灾害预警系统等应用开始进入大众日常生活。全球物联网保持高速增长，物联网领域具备巨大的发展空间。根据 GSMA 发布的《The mobile economy 2020 (2020 年移动经济)》报告显示，2019 年全球物联网总连接数达到 120 亿，预计到 2025 年，全球物联网总连接数规模将达到 246 亿，年复合增长率高达 13%。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 LiteOS[1]、亚马逊的 FreeRTOS[2]、Linux 基金会的 Zephyr[3]等。这些操作系统都有着不同的特点和优势，但它们都可以为物联网设备提供更好的支持和保障。随着物联网技术的不断发展，物联网操作系统也将会得到更广泛的应用和推广。

然而，物联网和嵌入式操作系统面临着严峻的安全挑战[4]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为“Mirai”的恶意软件[5]利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。俄乌冲突期间，乌克兰遭遇了俄罗斯发起的破坏性网络攻击，该攻击利用一种名为 Industroyer 的恶意软件[6]控制电力系统中的开关和断路器，意图瘫痪乌克兰的电力系统。造成上述这些安全事件发生的罪魁祸首就是嵌入式系统存在的安全漏洞。这些安全事件的不断发生也表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

当前物联网操作系统大多运行在基于 Arm Cortex M，RISC-V，MIPS 等架构的微控制器（Micro Control Unit MCU）的嵌入式设备上，任何执行控制或自动化功能的现代系统都可能包含使用物联网操作系统固件的微控制器。不幸的是，由

于这些微控制器系统具有与硬件紧密集成的特性，分析其固件的安全漏洞通常是一项具有挑战性的任务。

如果选择在硬件平台上进行固件漏洞分析工作，我们就需要在硬件上执行固件的过程中执行几乎所有的固件动态分析工作，而执行检查功能通常由硬件上的调试端口提供，这种端口通常仅限于少数几个断点，并不能很好地满足漏洞分析工作的需求。同时，当前的自动化的漏洞测试技术（如 fuzzing）需要运行设备的许多实例，以便有效地发现漏洞，对硬件的要求也会增加了漏洞分析的费用开销，从而降低自动化分析应用的规模。因此，开发自动化的固件漏洞测试工具的一个重要难题就是如何克服固件分析的硬件依赖问题。

通过固件仿真，亦称固件托管（Firmware Re-Hosting）技术来模拟嵌入式系统，使包含嵌入式系统的固件在商用计算机和服务器的软件模拟器上运行，是一种已被广泛证明了的，可以克服当前自动化的漏洞测试技术固件分析过程中的硬件依赖问题的有效方法。然而，当前的固件仿真技术仍存在很多问题，这些问题主要聚焦在嵌入式设备的各种难以模拟的外设上，同时业界对基于固件仿真的完善且高效的物联网操作系统模糊测试工具的需求也越来越强烈。

在此背景下，本研究将主要致力于改进基于固件仿真的物联网模糊测试方法，借助现有固件仿真与模糊测试技术，实现针对复杂物联网操作系统的模糊测试工具。

1.2 研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性[7]，对固件进行有效的安全测试是一项具有挑战性的任务。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了一些代表性的工具和系统，如 FIRMADYNE[8]、FUZZWARE[9]等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化[10]。下面

我将从固件仿真技术，嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[11]之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如，计时器、UARTs、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。为此，人们提出了许多不同的方法来应对这些挑战[12]。

1)　面向原始硬件外设访问的动态分析方案

一种方法是硬件在环（Hardware in the loop）动态分析方法，这种方法的核心思想是将固件分析过程中对外设的访问操作被转发到物理硬件中，由实际物理设备处理无法仿真的外设请求。利用这种方法的的工具有 Zaddach 等人提出的混合仿真系统 Avatar[13]，以及 Marius Muench 等人提出的 Avatar2[14]等，上述的二者均采用模拟器（QEMU)和真实硬件进行混合执行的方法。这些工具控制模拟器端完成固件指令的翻译和执行，并将 I/O 操作引导到真实设备端完成。

硬件在环方法是一种折衷的方案，尽管它可以提供有效利用现有漏洞挖掘技术的仿真环境，也可以使用实际物理设备来处理无法仿真的外设请求。但是，这种方案还是依赖于实际的物理设备，而仿真环境和物理设备之间的来回切换，也会影响到动态分析技术的运行速度和效果。

2)　基于原始硬件的外设建模方案

解决外设依赖的一种方法是像在 QEMU 中那样在底层内存映射寄存器接口上实现硬件。而这是一项费力的任务，扩展性不好。同时，外设种类与多样性也一直在不断丰富，而人工构建这些外设的仿真模型需要付出很高的成本。当前主流的仿真器 QEMU 也仅能仿真 10+种的设备，这对于固件仿真工作的需求是远远不够的。

有些工作采用自动化方法实现外设建模，这种方法的主流方案是通过观察未知外设的访问模式来自动建模，然后当固件模拟器访问未知外设寄存器时利用启发式计算产生适当的响应。Gustafson E 人提出了 Pretenter[15]工具。该工具通过观

察固件物理设备之间真实发生的交互记录，使用机器学习，模式识别等方法为每一个外设构建模型，再结合系统仿真器（如 QEMU 等）或程序分析引擎（如 angr）等，实现固件仿真与固件分析。

3) 基于程序分析的外设建模方案

另外一些方案是基于程序分析来对外设进行建模。通过对固件代码进行静态或动态分析，提取出固件与外设之间的交互逻辑，然后构建相应的外设模型，以替代真实的硬件外设。

Bo Feng 等人提出了 P2IM[16]工具，其基本实现原理是通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，并构建一个与真实设备相似的虚拟模型，从而实现对固件的仿真和分析。Cao C 等人提出了 Laelaps[17]，它在正常运行过程中因访问未实现的外设而卡住时，切换到符号执行，以找到正确的输入，从而引导 QEMU 找到与实际执行最相近的路径，完成固件模糊测试工作。Wei Zhou 等人提出了 uEmu[18]工具，利用符号执行方法，对固件镜像进行分析，将未知的外设寄存器表示为符号，构建推断规则，以响应未知的外设访问，并将产生的规则存储在知识库中。在动态固件分析期间引用知识库。最近的相关工作是 Tobias Scharnowski 等人提出的 Fuzzware[9]工具，该工具使用精确的 MMIO 建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，Fuzzware 可以自动生成模型，帮助工具将模糊过程集中在重要的输入突变上，提高输入的模糊测试输入数据的有效性。

4) 基于系统或驱动层替换的外设仿真方案

一种有前途的方法是基于系统或驱动层替换的仿真技术，或称为高级仿真(High Level Emulation, HLE)技术。它利用固件中的公共抽象来消除为外设提供低级支持的需要。

Chen D D 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[19], 通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失性内存、动态文件等仿真技术难题。Wenqiang Li 等人创建了 Para-rehosting[20]工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利用 x86 上许多现有的复杂软件测试工具对代码进行

测试。然而，这种方案的实现需要分析师获得固件样本的源代码，而在第三方的固件分析工作中，源代码的获取往往是困难甚至完全不可行的。Clements 等人提出了 HALucinator[21]工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abastract Layer HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真，这项工作在重托管简单的裸金属固件程序时取得了很好的效果。

综上，我调研了当前固件仿真技术的四个主要方向的研究现状，四种方案的优缺点对比如表 1.1 所示。

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于真实硬件的外设访问方案	Avatar, Avatar2	直接在真实硬件上进行测试，保证了测试的真实性和准确性	需要对真实硬件进行控制和监测，工作量大，成本高，测试速度慢，无法完全摆脱硬件依赖
基于程序分析的外设建模方案	P2IM, uEMu, Laelaps, Fuzzware	可以根据程序本身的信息自动推断出外设的行为，减少人工干预，提高通用性	难以完全覆盖所有外设的行为，可能存在误报或漏报
基于原始硬件的外设建模方案	QEMU, Pretender,	可以对外设的行为进行精确的模拟，提高测试效率和覆盖率	需要对每种外设进行专门的建模，工作量大，通用性差，有些外设行为难以模拟
基于系统或驱动层替换外设仿真方案	HALucinator, Firmadyne,	可以在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	需要对系统或驱动层进行修改和维护，可能引入新的错误或漏洞，对复杂操作系统的驱动层没有很好支持

在这些方案中，基于系统或驱动层替换的固件仿真方法是一种比较有前景的方案，它从固件系统层和驱动函数入手来解决的思路很好地解决了其他方案对复杂外设难以完全模拟的痛点利用这种方案的工具如 HALucinator 已经取得了一定的成果。但是，对于复杂的物联网操作系统来说，它们的系统库函数与驱动层的耦合更加复杂 ，同时有些操作系统对外设的操作会绕过对 HAL 函数的使用， 转而使用操作系统自身的设备驱动库函数。为解决当前的固件仿真策略并没有实现对复杂操作系统的驱动函数的支持的问题，成功完成对这些涉及物联网操作系统的固件的重托管工作，使固件可以被正确模拟，我们需要系统库与驱动库的函数

进行分析与替换，并完善模拟仿真工具对操作系统所必要的 MCU 硬件机制的支持。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷[22]。目前，嵌入式系统模糊测试技术主要从以下三个方向进行发展：

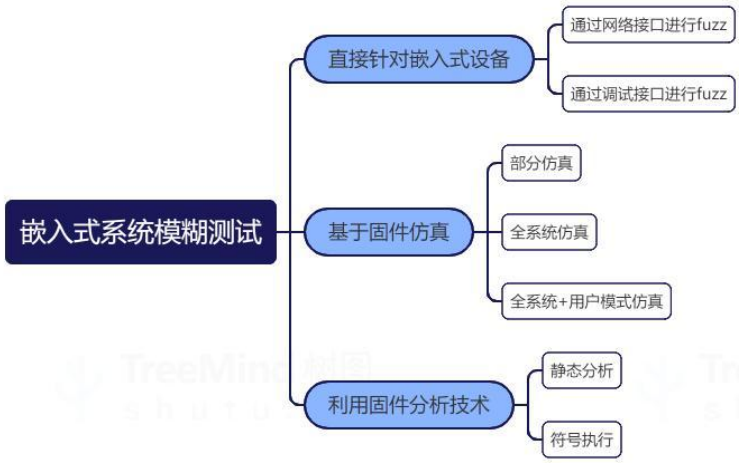

图 1.1 嵌入式系统模糊测试技术分类

1) 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Jiongyi Chen 等人开发的 IoTFuzzer[23]工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

2) 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[9]，uEmu[18]，HALucinator[24]等。这种方法

的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

3)　基于静态分析辅助的模糊测试方案

固件分析是一种辅助模糊测试的技术，它通过对嵌入式设备的固件进行静态或动态分析，获取固件的结构、功能、协议等信息，然后根据这些信息生成更有效的测试用例，提高模糊测试的针对性和有效性。例如，Firmalice[25]便是利用固件分析技术进行模糊测试的工具。其有三个主要组成部分：静态程序分析模块、符号执行和认证旁路检查模块。静态程序分析模块开发固件的程序依赖图，并创建从起始点到目标点的执行路径切片。符号执行引擎找到到达目标点的执行路径和满足约束的输入。认证绕过检查模块确认输入是否导致认证绕过漏洞。这种技术的优点是可以利用固件中已有的数据样本或语法规则，生成更符合设备输入要求的测试用例，触发更多的程序路径和异常，因此常被用于白盒模糊测试方案中。缺点是需要对固件进行深入的分析，可能需要专业的知识和工具，而且分析结果可能不完整或不准确。

综上，我调研了当前嵌入式系统模糊测试技术的三个主要方向的研究现状，三个方向的方案的优缺点对比如表 1.2 所示。

表 1.2 嵌入式系统模糊测试方案对比

方案	典型工具	优点	缺点
面向真实设备的黑盒模糊测试	IoTFuzzer	不需要目标设备的源码或固件，可以直接对网络接口进行测试，发现潜在的安全漏洞	测试效率低，难以覆盖所有的输入空间，需要对设备进行逆向分析和协议重构
基于固件仿真的灰盒模糊测试	Fuzzware, uEmu, HALucinator	可以利用固件镜像在虚拟环境中进行测试，不需要真实设备，可以获取代码覆盖率和崩溃信息，提高测试效率和精度	需要对固件进行解包和重打包，可能存在仿真不完全或不准确的问题，需要适配不同的硬件平台和操作系统
基于静态分析辅助的模糊测试方案	Firmalice	可以利用静态分析技术识别固件中的关键函数和变量，生成更有效的测试用例，提高代码覆盖率和漏洞发现率	需要对固件进行反汇编和符号执行，可能存在误报或漏报的问题，需要解决符号执行中的路径爆炸和约束求解问题

1.3 研究内容与章节安排

本文研究了当前基于设备仿真技术的物联网设备固件模糊测试工具，优化了针对物联网操作系统的固件仿真技术，完善基于高级仿真的模糊测试工具对 RTOS 操作系统的支持，实现了针对物联网 RTOS 操作系统的固件模糊测试平台。

第二章介绍了实现方案所需的背景知识，包括基于 ARM 微控制器的物联网设备硬件相关知识，RTOS 操作系统相关知识，嵌入式设备的模拟仿真技术，模糊测试技术等。

第三章介绍了物联网操作系统模糊测试工具的实现方案，此方案基于固件高级仿真技术，具有易于使用、可扩展性高、测试效果好等优点。

第四章对模糊测试工具进行测试与评估。首先介绍了对模糊测试工具的测试方案，之后从固件模拟能力与模糊测试能力两个方面对工具进行了测试与评估，证明了其优越性。

第五章总结了本文所做的工作，并对未来可做的进一步工作进行了展望。

第二章 背景知识

本章介绍了物联网操作系统模糊测试工具的相关背景知识，包括物联网操作系统的概念和分类，基于 ARM 微控制器的物联网设备硬件基础知识，模拟仿真平台的原理和特点，以及模糊测试技术的简介和优势。这些背景知识为本文的研究提供了必要的理论基础和技术支持，帮助读者更好地理解本文的工具设计和实现。

2.1 物联网操作系统背景知识

物联网操作系统是一种专门为物联网设备设计的软件平台，它可以提供设备管理、网络连接、数据处理等功能[26]。根据物联网操作系统的复杂程度可以将其划分以下三种类型：

基于 Unix 的物联网操作系统：这种操作系统是在传统的 Unix 或 Linux 系统上进行定制和优化，以适应物联网设备的资源限制和性能要求。基于 Unix 的物联网操作系统具有成熟的网络协议栈、文件系统、安全机制等特点，适合用于较复杂的物联网应用场景，如智能家居、工业控制等。例如，Android Things、Ubuntu Core、Raspbian 等都是基于 Unix 的物联网操作系统。

基于 RTOS 的物联网操作系统：这种操作系统是在实时操作系统（RTOS）的基础上开发的，它可以保证实时性、可靠性和低功耗等特点，适合用于对时间敏感和对资源紧张的物联网设备，如传感器、控制器等。例如，FreeRTOS、Zephyr、LiteOS 等都是基于 RTOS 的物联网操作系统。

裸机物联网操作系统：这种操作系统是直接在硬件上运行的，没有操作系统层，只有一些基本的驱动和库函数。裸机物联网操作系统最简单、最轻量，适合用于极端低功耗和极端低成本的物联网设备，如智能标签、智能卡等。例如，Arduino、mbed OS 等都是裸机物联网操作系统。

本文的研究主要是针对 RTOS 操作系统的固件仿真及模糊测试方案，因此本节重点介绍基于 RTOS 的嵌入式操作系统。RTOS 操作系统是一种实时操作系统，它能够及时响应外部事件或数据，并在规定的时间内完成任务。RTOS 操作系统的工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。RTOS 操作系

统的应用非常广泛，它可以提高系统的效率、可靠性和安全性，适用于各种嵌入式设备。

接下来，我将以 FreeRTOS、Zephyr、LiteOS 这三个常见的 RTOS 操作系统为例，简要介绍 RTOS 操作系统的结构。

2.1.1 FreeRTOS 操作系统

FreeRTOS 是一种实时操作系统（RTOS），专为微控制器和小型微处理器而设计。它是在 MIT 开源许可下免费分发的，包括一个内核和一组适用于各个行业领域的物联网（IoT）库。FreeRTOS 的结构如图 2.1 所示，主要由以下几个部分组成：

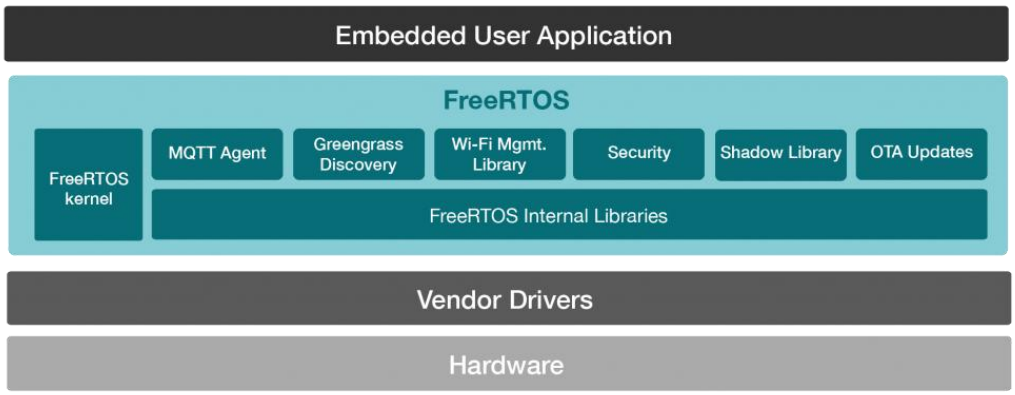

图 2.1 FreeRTOS 结构图

1) **FreeRTOS 内核：**内核是 FreeRTOS 的核心部分，负责任务管理、时间管理、内存管理、中断管理等功能。内核具有高可靠性、小占用空间和广泛的设备支持，是微控制器和小型微处理器的事实标准。
2) **FreeRTOS 集成：**集成是 FreeRTOS 与其他平台或服务的连接部分，如 AWS IoT、LoRaWAN、TF-M 等。集成可以让 FreeRTOS 更容易地实现物联网应用，FreeRTOS 也为各种集成提供了一些预配置的演示和参考实现。
3) **FreeRTOS 内部库：**库是 FreeRTOS 的扩展部分，提供了一些常用的功能和服务，如 TCP/IP 协议栈、命令行接口、文件系统、JSON 解析、MQTT 客户端等。这些扩展库可以根据用户需要选择使用。为了适配各开发商的驱动层，这些内部库中大多有着与驱动函数紧密耦合的 Portable 层，FreeRTOS 的 Portable 层的接口函数，隐藏底层硬件交互的驱动函数的细节，使应用程序可以专注于

上层业务逻辑。而开发商驱动函数与部分 FreeRTOS 内部库的 Portable 函数的高度抽象底层功能的特性，使利用高级替换解决固件仿真的硬件依赖功能成为可能。

2.1.2 LiteOS 操作系统

Liteos 是一种轻量级的嵌入式操作系统，主要用于物联网和智能终端设备。Liteos 的结构如图 2.2 所示，其大体上分为三层：内核抽象层（Kernel Abstract Layer 内核抽象层），中间件层和内核层。

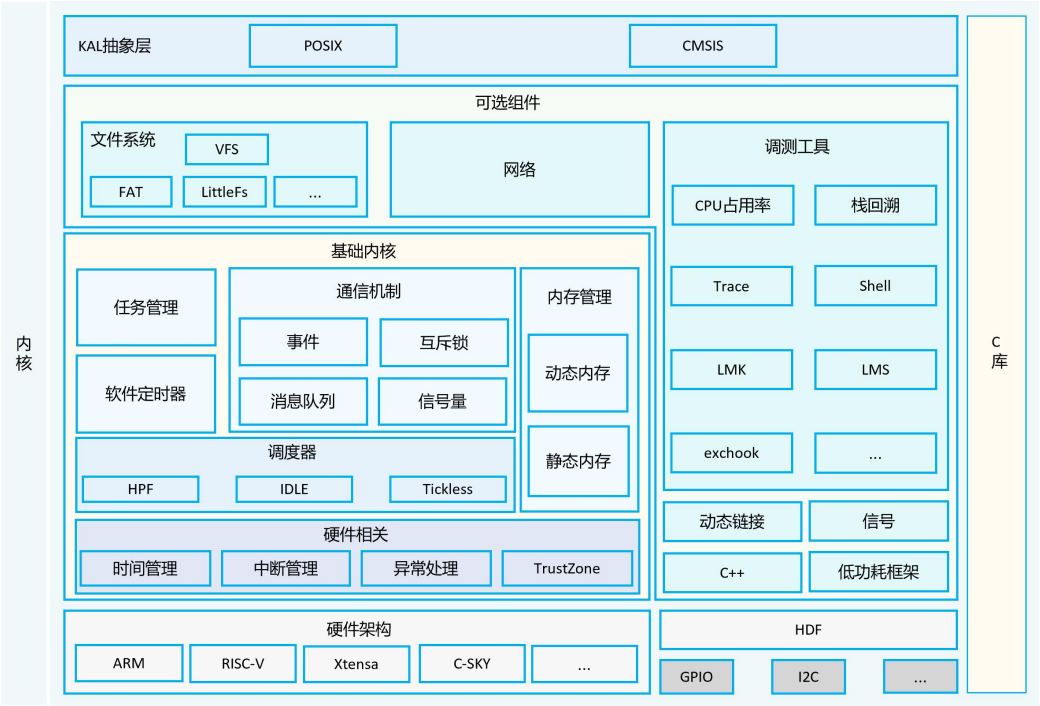

图 2.2 LiteOS 结构图

1) 内核层

内核层是 LiteOS 的核心部分，包括不可裁剪的极小内核和可裁剪的其他模块。极小内核包含任务管理、内存管理、中断管理、异常管理和系统时钟等基础功能。可裁剪的模块包括信号量、互斥锁、队列管理、事件管理、软件定时器等增强功能。内核层还支持 UP（单核）和 SMP（多核）模式，即支持在单核或者多核的环境上运行。内核层还提供了调试组件，如 shell 命令、Trace 事件跟踪、获取 CPU 占用率等，以帮助用户定位和解决问题。

2) 内核抽象层

内核抽象层是 LiteOS 的硬件平台和系统内核等与应用层之间的接口，提供了一套统一的系统库标准接口，包括 CMSIS，POSIX 等，以提升操作系统的可移植性和兼容性。通过内核抽象层，应用层程序可以适应 Cortex-M series、Cortex-R series、Cortex-A series 等多种芯片架构，忽略其底层差异，可以快速移植到多种硬件平台上。

3) 中间件层

中间件层是 LiteOS 提供的一系列组件，以支持多种物联网应用场景和功能，包括网络协议栈、文件系统、OTA、GUI、AI、传感框架等。中间件层提供了丰富的网络协议栈，如 CoAP/LwM2M、MQTT 等，以实现设备与云端的通信和管理。中间件层还提供了一套轻量级的文件系统接口，支持 ramfs、fatfs 等文件系统。此外，中间件层还提供了 OTA、GUI、AI、传感框架等业务组件，以支持设备的远程升级、图形界面、人工智能和传感器数据采集等功能。这些中间件层往往会负责与底层驱动的交互的工作，例如文件系统的中间件需要与底层 SD 卡，Flash 闪存卡进行交互。

中间件层是本文对 LiteOS 研究工作的重点，通过分析中间件层与底层硬件驱动函数的耦合关系，使用高级替换方案抽象并替换部分中间件层与硬件驱动层的关键函数，可以有效解决 LiteOS 固件样本的硬件依赖问题。

2.1.3 Zephyr 操作系统

Zephyr 是一款面向物联网设备的开源实时操作系统，由 Linux 基金会管理，支持多种处理器架构和开发板。Zephyr 的结构可以分为三层：kernel，os services 和 application services。

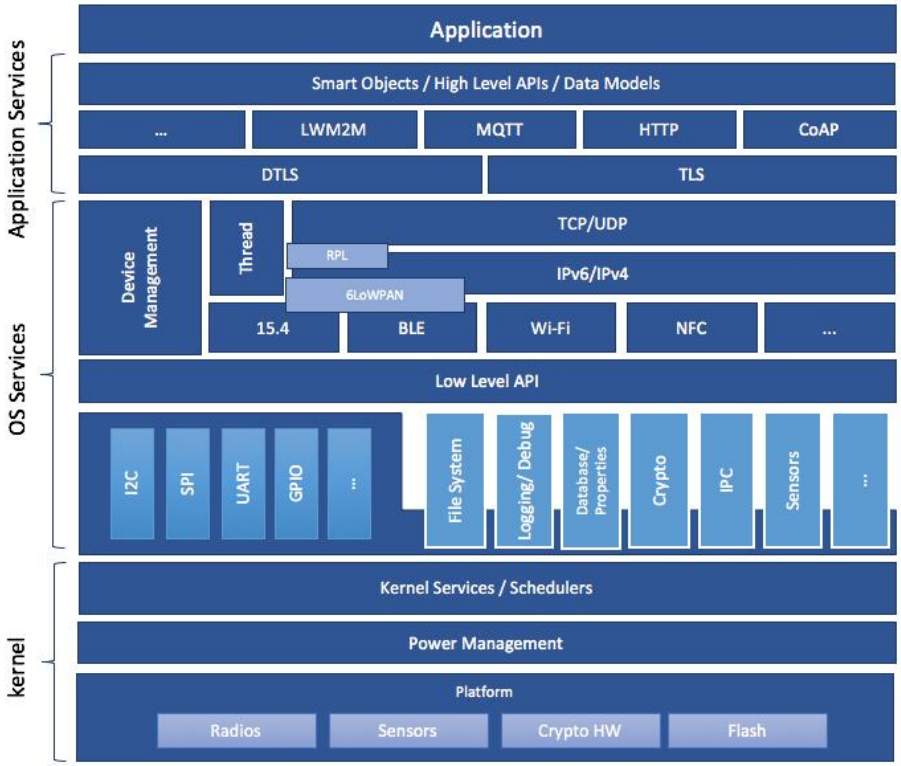

图 2.3 Zephyr 系统架构图

1) kernel 层是 Zephyr 的核心，提供了基本的调度、同步、中断、内存管理等功能，以及一些可选的内核服务，如用户态/内核态隔离、堆栈溢出检测、可信执行环境等。kernel 层还定义了一套硬件抽象层（HAL），方便移植到不同的硬件平台上。

2) os services 层是 Zephyr 的扩展，提供了一些常用的操作系统服务，如文件系统、网络协议栈、蓝牙协议栈、设备驱动框架、加密框架等。os services 层可以根据应用的需要选择性地编译和链接。

3) application services 层是 Zephyr 的应用接口，提供了一些高级的服务和库，如 shell、logging、测试框架、JSON 解析器等。application services 层也可以根据应用的需要选择性地编译和链接。

2.2 基于 ARM 微控制器的物联网设备硬件基础知识

本文的研究主要针对运行在 Arm Cortex-M 系列微处理器的 RTOS 操作系统，接下来我将介绍基于 Arm 微控制器的物联网设备硬件基础知识。

ARM Cortex-M 是 ARM 架构处理器核心中，低端系列的统称，由 Arm 公司所

授权。这组核心的特点为低成本以及高能源效率的微处理器优化设计，已有上千万个消费性设备中有此系列的微处理器。ARM Cortex-M 系列微处理器采用了 Thumb-2 指令集，提供了 32 位的性能和 16 位的代码密度[27]。同时该系列提供了一些特殊的功能，如嵌套向量中断控制器（Nested vectored interrupt controller NVIC）、系统计时器（System Tick Timer Systick）、位带存储器（Bit-Band Memory Bit-Band）、存储器保护单元（Memory Protect Unit MPU）等，可以提高系统的可靠性和实时性 。

2.2.1 Arm Cortex M 架构特性

Arm Cortex M 架构常用特性有 NVIC，Systick，主栈与进程栈等，这些特性常被嵌入式操作系利用以实现任务调度、中断管理、时间管理和内存管理等功能。

1) 嵌套向量中断控制器

嵌套向量中断控制器简称 NVIC，它是 Arm Cortex M 架构的一个核心组件，负责处理外部和内部的中断请求。RTOS 操作系统可以通过配置 NVIC 的优先级寄存器和使能寄存器，来设置不同中断源的优先级和使能状态，从而实现中断的响应和屏蔽。同时，RTOS 操作系统可以通过读取 NVIC 的挂起寄存器和活动寄存器，来判断当前是否有中断发生，以及哪些中断正在执行。这些信息对于任务调度和中断嵌套是非常重要的。

2) 系统定时器

系统定时器又称 Systick，它是 Arm Cortex M 架构的另一个核心组件，负责提供一个可编程的周期性中断。RTOS 操作系统通过配置 Systick 的重装载寄存器和控制寄存器，来设置 Systick 的时钟源、计数模式和中断使能。通常，RTOS 操作系统会将 Systick 设置为产生一个固定频率的中断，比如 1ms 或 10ms，作为操作系统的心跳。这样，RTOS 操作系统就可以利用 Systick 中断来实现时间管理和任务切换等功能。

3) 栈模式切换过程

主栈与进程栈是 Arm Cortex M 架构支持的两种栈模式。主栈是默认的栈模式，用于执行异常处理程序（包括复位、NMI、硬件错误和 SVC 等）。进程栈是可选的栈模式，用于执行线程模式下的用户程序。RTOS 操作系统可以通过配置控制寄存

器的 SPSEL 位，来选择当前使用哪种栈模式。一般来说，RTOS 操作系统会在启动时初始化主栈指针和进程栈指针，并将 SPSEL 位设置为 1，以使用进程栈模式。这样，RTOS 操作系统就可以为每个任务分配一个独立的进程栈空间，并在任务切换时保存和恢复相应的栈指针。栈模式的切换是实现多任务并发和内存保护的基础。

2.3 模拟仿真平台

模拟仿真平台是一种可以在不同的硬件和软件环境下运行目标程序的工具，当前主流的模拟仿真平台有 QEMU，Unicorn[28]，Qiling[29]等。模拟仿真平台是固件仿真工具的重要部分，固件仿真工具依赖这些仿真平台提供的动态译码等功能，来获取与真实硬件设备类似的运行环境。本文的工具基于 Unicorn 模拟仿真平台实现，而 Unicorn 重用了 QEMU 的 CPU 仿真组件作为其核心。下面我将首先简要介绍 QEMU 模拟器，之后详细介绍 Unicorn 引擎的原理及特点。

2.3.1 QEMU 模拟器

QEMU 即 Quick EMUlator，它是一种通用且开源的计算机仿真和虚拟化程序，它可以在任意支持的架构上为任何机器运行一个完整的操作系统，也可以在任意支持的架构上为另一个 Linux 用户运行一个用户模式的仿真系统。QEMU 还可以与基于内核的虚拟机（KVM）进行互操作，以接近本机运行的速度运行虚拟机。QEMU 还可以对用户级进程进行仿真，允许为一种体系结构编译的应用程序在另一种体系结构上运行。QEMU 可以被用作仿真器，超级监视器等。作为仿真器使用时，它允许我们运行由于兼容性原因而无法在当前操作系统上运行的软件。作为超级监视器使用时，它允许我们创建并管理虚拟机。

QEMU 有两种主要的仿真模式：用户模式和系统模式。用户模式可以让一个进程在另一个架构的 CPU 上执行，系统模式可以仿真整个系统，包括处理器和外设。

2.3.2 Unicorn 引擎

Unicorn 引擎是一个轻量级、多平台、多架构的 CPU 模拟器框架，可以代替 CPU 模拟代码的执行，它的设计目的是作为一款轻量级的 CPU 模拟器，解决分析师们在二进制分析中遇到的问题，例如：如何在不同的架构上运行二进制文件、如何在运行时修改二进制文件等等。目前 Unicorn 引擎可以模拟多种架构，包括 Arm、Arm64（Armv8）、M68K、Mips、Sparc 和 X86（包括 X86_64）等。常用于二进制分析、漏洞挖掘、程序虚拟、恶意代码分析、Fuzzing 等。

1) 实现原理

Unicorn 引擎是一款基于 qemu 的轻量级多架构模拟器，它使用了 qemu 的翻译引擎（TCG），将目标指令集翻译成中间表示（IR），然后再翻译成主机指令集，从而实现跨平台的模拟。同时，Unicorn 引擎在 qemu 的基础上进行了一定的简化与修改，使其更适合于关注 CPU 操作而非全系统仿真。

1. Unicorn 引擎对 qemu 的内存管理进行了简化，它只使用了一个连续的内存空间，而不是 qemu 的分段内存模型。这样可以减少内存开销和管理复杂度，也方便了用户对内存的操作和访问。
2. Unicorn 引擎对 qemu 的设备模拟进行了裁剪，它只保留了最基本的设备，如 CPU，内存，中断控制器等，而去掉了其他不必要的设备，如硬盘，网卡等。这样可以提高模拟的速度和灵活性，也避免了一些设备相关的问题和错误。
3. Unicorn 引擎提供了一套简洁的 API，它可以让用户方便地创建和配置模拟器，执行和控制代码，读写寄存器和内存，设置和处理中断和异常等。Unicorn 引擎还支持多种编程语言，如 C，Python，Java 等。

2) 接口设计

Unicorn 引擎提供了一套简洁易用的 API 接口，让我们可以使用不同的编程语言来控制模拟器的行为。Unicorn 引擎的 API 接口主要分为以下几类：

1. 初始化和销毁：这些接口用于创建和释放模拟器实例，指定硬件架构和模式。
2. 内存管理：这些接口用于映射和取消映射模拟器的内存空间，以及读写内存中的数据。
3. 寄存器操作：这些接口用于读写模拟器中的寄存器值。
4. 模拟控制：这些接口用于开始和停止模拟执行，以及设置模拟执行的时间和指

令限制。

5. 钩子函数：这些接口用于注册和注销钩子函数，以便在模拟执行过程中监听和处理不同类型的事件，如指令执行，内存访问，系统调用等。
6. 查询状态：这些接口用于查询模拟器的当前状态，如错误码，版本号等。

3) 钩子函数

Unicorn 引擎提供了一系列的钩子函数相关 API 接口，用于在模拟执行过程中监控和修改内存、寄存器、指令等状态。Unicorn 引擎支持多种类型的钩子函数，如内存读写钩子、内存映射钩子、代码执行钩子、中断处理钩子等，每种钩子函数都有特定的作用和触发条件。

钩子函数的注册和注销机制：Unicorn 引擎提供了 uc_hook_add 和 uc_hook_del 两个 API 接口，用于注册和注销钩子函数。注册钩子函数时需要指定钩子类型、回调函数、用户数据和地址范围等参数。注销钩子函数时只需要传入注册时返回的钩子句柄即可。

钩子函数的回调参数：Unicorn 引擎在触发钩子函数时会将一些相关信息作为参数传递给回调函数，如模拟器对象、事件类型、地址值、数据值、用户数据等。回调函数可以根据这些参数进行相应的处理，如打印日志、修改内存、停止模拟等。

2.4 模糊测试

2.4.1 模糊测试技术简介

模糊测试技术是一种软件测试方法，它通过向目标系统输入随机或无效的数据，观察系统是否出现异常行为，从而发现潜在的安全漏洞或缺陷。模糊测试技术的原理是基于这样一个假设：如果一个系统可以正确地处理任意的输入数据，那么它就是一个健壮和安全的系统。模糊测试技术可以有效地发现软件中的安全漏洞和逻辑错误，提高软件的可靠性和安全性。

模糊测试技术的发展可以分为三个阶段：第一代模糊测试技术是基于黑盒测试的，它不需要知道目标系统的内部结构和逻辑，只需要生成随机的输入数据，并监测系统的输出和状态。第一代模糊测试技术的代表工具有 zzuf，radamsa，Peach

等。第二代模糊测试技术是基于白盒测试的，它需要知道目标系统的源代码或二进制代码，利用静态分析或动态分析的方法，生成更有针对性和覆盖率更高的输入数据，并利用反馈机制来指导输入数据的生成。第二代模糊测试技术的代表工具有 AFL，libFuzzer，honggfuzz 等。第三代模糊测试技术是基于灰盒测试的，它结合了黑盒测试和白盒测试的优点，不需要完全知道目标系统的内部结构和逻辑，但可以利用一些辅助信息，如符号执行，污点分析，约束求解等，来生成更有效和智能的输入数据，并利用多种反馈机制来优化输入数据的生成。第三代模糊测试技术的代表工具有 Driller[30]，QSYM[31]，Angora[32]等。

2.4.2 嵌入式设备模糊测试发展现状

对嵌入式设备的模糊测试发展是一个新兴且具有挑战性的领域。嵌入式设备通常具有多种通信接口和协议，如串口，USB，蓝牙，Wi-Fi 等，这些接口和协议可能存在各种安全漏洞或缺陷。对嵌入式设备进行模糊测试的研究涉及嵌入式设备的固件或代码的获取、嵌入式设备的逆向工程或分析工作、合适的模糊测试工具和方法的选择、有效的对嵌入式设备的监控和反馈机制、嵌入式设备的复位或重启问题的处理等多个方向的内容：。目前对嵌入式设备进行模糊测试的工具还比较少，一些常见的工具有 Firmadyne，Firm-AFL[33]等。

2.4.3 模糊测试工具 AFL

American fuzzy lop[24]（简称 AFL）是一款安全导向的模糊测试工具，它利用一种新颖的编译时插桩和遗传算法，自动发现能够触发目标二进制文件中新的内部状态的干净、有趣的测试用例。这样可以显著提高被测试代码的功能覆盖率。该工具生成的紧凑的合成语料库也可以用于后续的其他更耗时或耗资源的测试方案。

AFL 在模糊测试领域有着很高的地位，它是在 2013 年出现的，它的诞生是一个里程碑标志，对之后出现的模糊测试工具产生了重大影响。 它不仅能够发现真实环境中的漏洞，而且具有很高的性能和稳定性，几乎不需要任何配置和调优，能够无缝地处理复杂、实际的用例。

1) AFL 实现原理

AFL 利用了一些创新的技术来提高模糊测试的效率和覆盖率。AFL 实现原理示意图如图 2.4 所示。AFL 的实现原理主要包括以下几个方面：

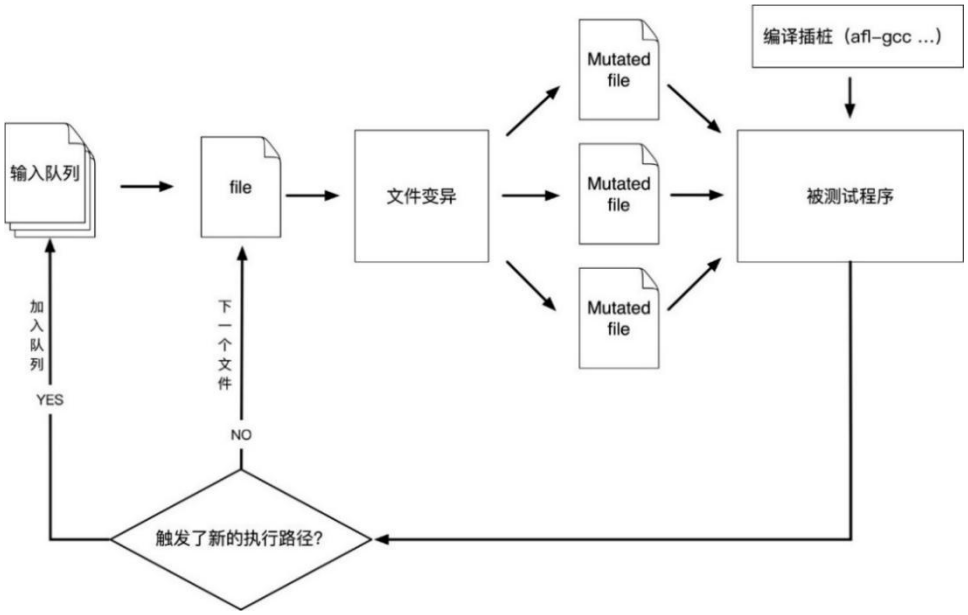

图 2.4 AFL 模糊测试原理图

输入队列：AFL 维护了一个输入队列，用来存储已经生成的输入文件，这些文件被称为种子文件。AFL 会不断地从输入队列中取出一个种子文件，对其进行变异，然后将变异后的文件作为新的输入文件，送给目标程序执行。如果新的输入文件能够触发目标程序的新的执行路径，那么 AFL 会将其加入到输入队列中，作为新的种子文件。这样，AFL 可以不断地扩展输入队列，探索目标程序的更多执行路径。

执行路径触发：AFL 使用了一种轻量级的代码插桩技术，来监测目标程序的执行路径。具体来说，AFL 会在目标程序的每个基本块（basic block）的开头插入一段代码，用来更新一个共享内存区域中的一个计数器。这个计数器的值由当前基本块的编号和上一个基本块的编号异或得到。这样，AFL 可以通过读取共享内存区域中的计数器值，来判断一个输入文件是否触发了目标程序的新的执行路径。如果是，那么 AFL 会将该输入文件保存下来，并给予一定的奖励。

文件变异：AFL 使用了多种文件变异算法，来生成新的输入文件。这些算法包括：位翻转（bit flipping），字节翻转（byte flipping），字节随机替换（byte randomization），字节删除（byte deletion），字节插入（byte insertion），字节重复（byte duplication），块交换（block swapping），块删除（block deletion），块插入

（block insertion），块重复（block duplication），块重写（block overwriting）等。AFL 会根据每种算法产生新执行路径的概率，动态地调整算法的权重，优先使用效果较好的算法。

2) AFL 使用模式

AFL 有以下几个使用模式：

1. Instrumented mode 有源码插桩模式：这种模式下，需要使用 AFL 提供的编译器（如 AFL-gcc 或 AFL-g++）对源代码进行插桩编译，然后使用 AFL-fuzz 对编译后的程序进行模糊测试。

2. QEMU mode 无源码插桩模式：这种模式下，不需要源代码，只需要使用 AFL 提供的二进制插桩工具（如 AFL-dyninst 或 AFL-qemu-trace）对二进制文件进行插桩，然后使用 AFL-fuzz 对插桩后的程序进行模糊测试。它适用于无法获取源码或者无法编译的情况。

3. Unicorn 模式：这种模式下，不需要二进制文件，只需要使用 Unicorn 引擎（一个轻量级、可移植、多架构的 CPU 仿真器）来执行目标代码片段，并使用 AFL-fuzz 对 Unicorn 引擎进行模糊测试。这种模式适用于一些难以获取二进制文件，需要绕过一些检测或者处理复杂输入格式的文件，或者无法在常规环境下运行的目标，例如固件、虚拟机等。

3) AFL-Unicorn 模式

Unicorn 模式是 AFL 最新引入的一种模式，也是 AFL 中最特殊和最强大的一个模式，它可以实现对任意架构和任意平台的代码片段进行模糊测试，而不受操作系统和硬件环境的限制。它利用了 Unicorn 引擎提供的 API 来控制目标代码片段的执行流程和内存状态，并通过 AFL 提供的 Unicorn_mode 库来与 AFL-fuzz 通信。它可以让用户自定义一个 fuzzing harness，用来指定目标程序的哪些部分需要被模拟执行，以及如何处理输入数据。Unicorn 模式通过实现 AFL 的 qemu 模式中的块边缘检测来工作，利用 Unicorn 引擎提供的代码模拟和内存映射功能，将二进制代码加载到内存中，并记录代码覆盖率信息，从而指导输入样本的生成和变异。

Unicorn 模式的优点是可以对任意架构的二进制文件进行 fuzzing，而不受 QEMU 模拟器的限制；同时可以绕过一些防御机制，如 ASLR、NX、Canary 等；还可以对输入数据进行预处理或后处理，以适应复杂的输入格式。Unicorn mode 的缺点是需要用户编写 fuzzing harness，这需要对目标程序有一定的了解；而且由

于 Unicorn 引擎本身的性能问题，Unicorn mode 的 fuzzing 速度会比 instrumented mode 和 QEMU mode 慢很多。

要使用 Unicorn 模式，需要先安装 AFL 和 Unicorn 引擎，并编译 Unicorn_mode 目录下的代码。然后，需要准备以下内容：

1. 要进行模糊测试的二进制代码
2. 目标程序的内存映射和初始状态
3. 一些初始输入样本，作为输入队列的起点
4. 基于 AFL-Unicorn 的测试驱动程序，可以用 Python、C 或 Rust 编写，该驱动程序需要完成以下任务：
    1) 添加内存映射区域
    2) 将二进制代码加载到内存中
    3) 至少用 uc.emu_start 模拟一条指令以启动 AFL 模糊测试工具的 forkserver
    4) 从命令行指定的文件中加载和验证要进行模糊测试的数据
    5) 设置寄存器和内存状态，开始执行目标代码
    6) 如果检测到崩溃，通过抛出信号或在崩溃验证回调中指示崩溃

2.5 本章小结

本章介绍了实现基于固件仿真的模糊测试工具实现方案所需的背景知识。介绍了 ARM Cortex M 架构相关知识，重点介绍了 ARM Cortex M 架构在嵌入式操作系统中常用的特性。介绍了 RTOS 操作系统相关知识，使读者对物联网操作系统特性有一定了解。介绍了嵌入式设备的模拟仿真技术，模糊测试技术等技术的概念及原理，本文的方案是利用基于固件模拟仿真的模糊测试技术进行漏洞挖掘。

第三章 设计与实现

本章介绍本文基于固件仿真的模糊测试工具的设计与实现工作。本章首先简要介绍了工具的整体结构，之后对工具的三个主要模块的设计与实现工作做了详细说明。

3.1 整体结构设计

模糊测试系统的整体结构如图 3.1 所示，工具自上到下依次为固件模拟与中断控制模块，库函数替换模块，Fuzzer IO 模块。固件模拟与中断控制模块由 Unicorn 引擎与中断控制模块两部分构成，该模块负责加载并执行固件样本，同时该模块补充了 Unicorn 引擎缺失的中断控制机制，以完善 RTOS 操作系统正确模拟所依赖的处理器特性。库函数替换模块负责完善工具的固件仿真功能，解决硬件依赖问题，同时它与 Fuzzer IO 模块进行协作，使固件获取到模糊测试的输入。Fuzzer IO 模块与 AFL 工具对接，负责提供变异的模糊测试数据。

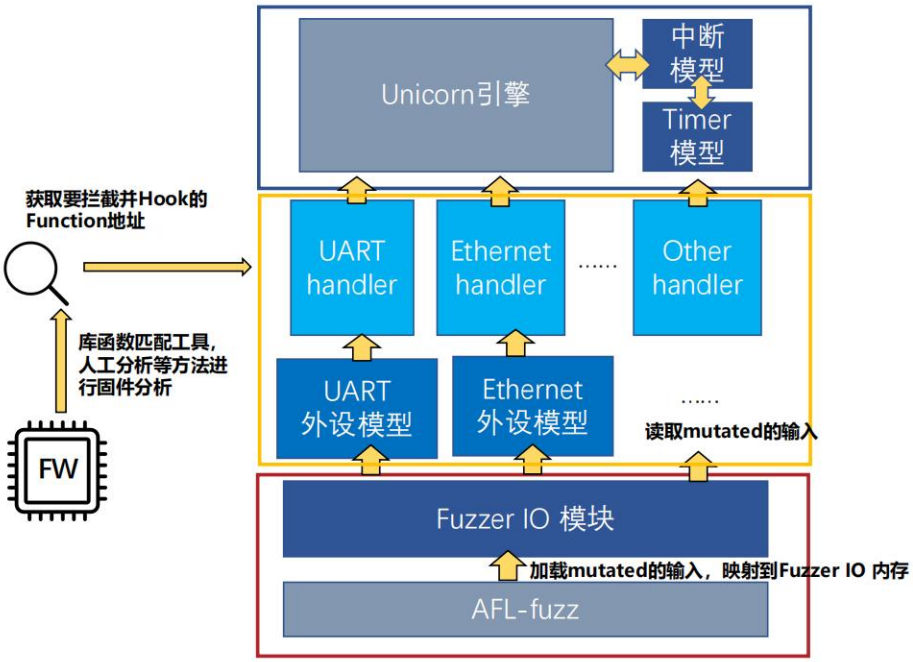

图 3.1 模糊测试系统整体结构

固件模拟与中断控制模块：该模块利用 Unicorn 模拟器框架对固件样本进行加

载和执行。同时模块补充了 Unicorn 引擎所缺失的中断控制与计时器模块。利用这个模块可以模拟绝大部分 Arm Cortex M 架构的 MCU 微处理器行为。而有些难以处理的功能如与外设的交互等则交给库函数替换模块进行处理并反馈。

库函数替换模块：该模块利用 Unicorn 模拟引擎的 Hook 机制，用我们自行编写的 handler 函数及其配套的对相关外设功能进行简单抽象的外设模型，替换固件样本中系统库与驱动层的关键函数，解决固件模拟的硬件依赖问题。同时它负责与 Fuzzer IO 模块进行协作，读取 Fuzzer 加载到内存中的输入，并分发到外设模型中。在固件样本运行到用于获取外部数据的驱动函数时，触发我们编写的 Handler 函数，之后 handler 函数会从外设模型中获取 Fuzzer 加载的数据，模拟驱动函数接收外部输入的行为，将数据映射到 Unicorn 引擎存储输入的的对应内存中，实现 Fuzzing 数据的获取。

Fuzzer IO 模块：该模块利用 Unicorn-AFL 工具接口，负责加载 AFL 产生的变异的输入到内存中，这些变异的输入可以提供给上层高级替换模块中的 handler 函数与外设模型进行读取。

接下来，我将分别介绍工具的固件模拟与中断控制模块，库函数替换模块以及 FuzzerIO 模块的设计，其中重点介绍工具的核心部分：库函数替换模块。

3.2 固件模拟与中断控制模块设计

一、固件模拟功能

固件模拟模块负责大部分对固件的模拟工作，它首先根据配置文件的内容来创建并初始化 Unicorn 对象与中断控制器，创建必要的内存映射，加载固件代码，并完成钩子函数的注册工作。之后，它加载 AFL 提供的突变输入到内存中。最后，它利用 Unicorn 引擎来执行固件指令，开启固件仿真过程。该模块的主要工作如下：

1. 创建 Unicorn 对象，并初始化中断控制模块。
2. 根据配置文件中的 memory_map 项，在 Unicorn 对象中创建并加载内存映射。
3. 根据配置文件中的 handlers 项，注册库函数替换模块的钩子函数，完成与库函数替换模块的对接。在固件仿真过程中被注册的钩子函数会在合适的地方被启用。
4. 根据配置文件加载运行过程的日志跟踪选项，选择性地启用内存读写跟踪，函

数调用跟踪，基本块执行跟踪等功能。这些功能通过 Unicorn 的钩子函数机制实现，被启动的功能对应钩子函数将注册到 Unicorn 引擎中并在固件仿真过程中被调用。

5. 调用 Fuzzer-IO 模块的 load_fuzz 函数，将突变的输入加载到内存中。
6. 利用 Unicorn 引擎的 emu_start 方法，开启固件仿真。

在固件模拟与中断控制模块中，利用 Unicorn 引擎可以模拟大部分的 CPU 行为。而 Unicorn 引擎中缺失的 RTOS 操作系统运行所必要的中断控制与计时器机制，则需要由中断控制子模块完成，下面我将简要介绍工具的中断控制模块的实现。

二、中断控制功能

当前主流的 RTOS 操作系统利用 Systick 来实现时钟节拍，利用中断机制来实现任务切换和中断响应。RTOS 还利用 Cortex-M 内核的特性，如双栈指针、异常返回等，来简化任务管理和上下文切换。为了成功在模拟平台上运行 RTOS 操作系统，一个功能完善的中断控制模块是不可或缺的，然而，Unicorn 引擎为了专注于 CPU 的执行，人为地删去了中断与计时器的概念，因此我们的工作需要自行构建一个中断控制模块，这个模块需要实现所有 RTOS 常用的 Systick 计时器和中断控制功能。该模块的主要功能由计时器与中断触发函数，以及中断退出处理函数完成，下面我简要介绍这两个函数的设计与实现。

1. 计时器与中断触发函数

为了更好地适应模糊测试工具的需求，系统定时器模型以执行基本块数量作为计量执行时间的标准，实现了“确定性”的计时器行为，在固定数量的程序基本块执行后触发 Systick 中断，来模拟系统时钟的行为。同时中断控制模型会通过轮询或随机的方式逐个启动已使能的中断服务程序，模拟现实世界的外设行为。

上述定时器的计时功能利用 Unicorn 的 HOOK_BLOCK 机制实现，使用 HOOK_BLOCK 注册的函数在每个基本块执行前被调用。计时器功能的主要逻辑由 NVIC_tick_check 函数来完成，该函数被触发时会依次完成以下几个步骤：

1. 通过检查 CPSR 寄存器的 PRI MASK 标志位，判断是否需要屏蔽可屏蔽中断，如果是则退出函数，否则进入步骤 2。
2. 当前已执行基本块数目加一。
3. 检查当前已经过的基本块数目，根据当前已执行的基本块数目判断是否执行以下两个任务：

a) 每执行 SYSTICK_ROUND 个基本块，检查 Sys tick 是否已使能，若使能，将 Sys tick 中断号置入挂起列表中，等待加入活跃队列中运行。

b) 每执行 INTR_ROUND 个基本块，从已使能中断的列表中以随机或轮询的方式将一个中断号放置到挂起列表中，等待加入活跃队列中运行。

4. 检查当前的挂起列表，如果列表不为空，且当前没有活跃的中断，进入步骤 4，否则退出函数。

5. 进入中断，具体步骤如下：

a) 移除中断号从挂起列表中的第一个元素，设置活跃中断号为该元素中断号。

b) 模拟 NVIC 的行为，具体步骤如下：

1. 保护现场，压栈部分寄存器信息。

2. 设置控制寄存器使 SP 寄存器的值完成 MSP 与 PSP 的切换。

3. 在 IPSR 寄存器中写入对应中断号。

4. 修改 LR 寄存器为 EXC_RETURN。

c) 修改 PC 寄存器，使模拟器跳转到对应中断请求函数中执行中断服务程序。

2. 中断退出处理函数

中断的退出则由 Unicorn 的 HOOK_INTR 机制负责，使用 HOOK_INTR 注册的函数在每个中断返回前被调用。实际上，由于中断触发时 NVIC 会将 LR 寄存器的值写为 EXC_RETURN，因此，在中断函数执行完毕后，如果 Unicorn 引擎发现当前的 PC 值大于 EXC_RETURN，它便可以判断当前发生了中断返回事件。中断退出函数的主要逻辑由 NVIC_intr_handle 函数完成。该函数被触发时会依次完成以下两个步骤：

1. 检查当前 PC 寄存器的值，如果大于等于 EXC_RETURN，进入步骤 2，否则退出函数。

2. 退出中断，具体步骤如下：

a) 设置活跃中断号为-1 代表没有正在活跃的中断处理程序。

b) 模拟 NVIC 行为，具体步骤如下：

1. 检查当前 PC 值，即被 Unicorn 引擎写入的 LR 的值的标志位，如果标志位是使用进程堆栈的线程模式，则设置控制寄存器使 SP 寄存器的值完成 PSP 与 MSP 的切换。

2. 恢复现场，将寄存器信息出栈，并写入对应寄存器中。

3. 将 IPSR 寄存器置零。

3.3 库函数替换模块设计

库函数替换模块实现了用于替换固件样本中系统库与驱动层的关键函数模块 handler 函数及其配套的外设模型，解决了固件模拟的硬件依赖问题，并与 FuzzerIO 模块对接，完成模糊测试数据的输入。

库函数替换模块利用 Unicorn 引擎提供的实用又强大的 Hook 机制，实现对函数的抽象和替换工作。其核心思想如图 3.2 所示。固件执行过程中，当发生指令跳转，程序计数器（Program Counter PC）的值命中我们想要 Hook 的函数首地址时，会触发 Unicorn 引擎的 UC_HOOK_BLOCK 机制，此时固件执行会暂时停止，转而执行我们编写的对应的 Hook 函数，并将当前 PC 位置内容覆写为 bx lr 指令。在 Hook 函数执行完成后，Unicorn 引擎将从停止的位置开始，继续模拟固件执行，而此时的固件 PC 指向位置对应指令已被覆盖为 bx lr，固件将不执行函数代码，转而执行此指令，并跳转到函数被调用位置执行接下来的指令。这样就在执行 Hook 函数的同时跳过了固件代码中原本对这个函数的执行。

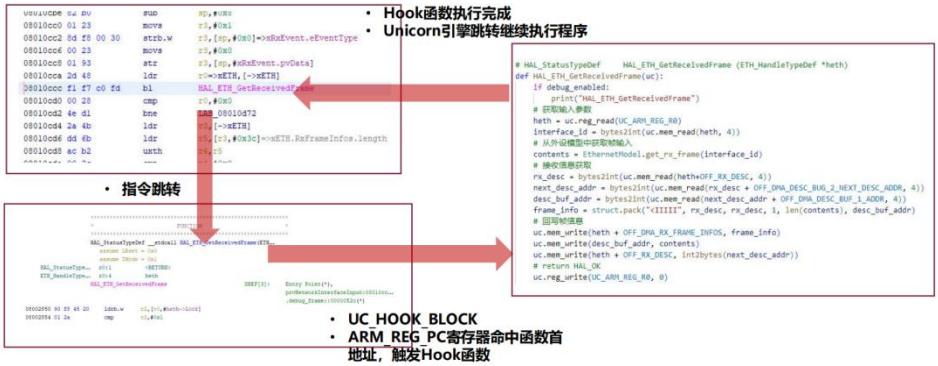

图 3.2 高级替换 Hook 机制示意图

接下来，我将从系统初始化，日志记录，中断回调函数，串口通信，网络通信，文件系统等几个方面，介绍我对 RTOS 操作系统的分析工作以及库函数替换模块的 Hook 函数设计工作。

一、系统初始化功能

由于模拟器没有提供底层外设硬件的支持，而嵌入式系统的初始化工作涉及到很多对硬件相关模块进行的初始化设置，当系统运行到这些初始化工作的模块

时，缺失的硬件支持会导致程序运行失败，这种失败一般表现为退出程序或者进入死循环。本工具的库函数替换模块针对三种类型的初始化失败问题，实现了不同的应对初始化函数的钩子函数实现方案，完成了固件系统初始化的功能。这三种方案如图 3.3 所示。接下来，我将分别介绍着三种方案的实现过程。

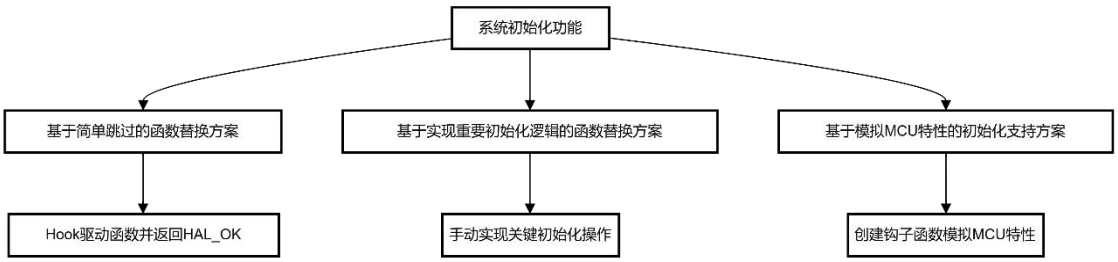

图 3.3 系统初始化功能实现方案分类

1) 基于简单跳过的函数替换方案

对于大部分这些导致初始化失败的驱动函数，可以选择 Hook 这个函数并直接返回一个 HAL_OK 指示值，而不关心其内部逻辑，这种简单方式进行绕过，简单的替换函数如表 3.1 所示。我们对系统驱动层的抽象并不需要维护这些固件外设的状态信息，因此大多时候这种简单的替换并不会对我们的固件仿真工作造成影响。

表 3.1 简单返回 HAL_OK 标志的函数

```
1. def return_HAL_OK(uc):
2.     """simple return for HAL_Init"""
3.     uc.reg_write(UC_ARM_REG_R0, HAL_OK)
```

这类函数的一个典型示例是在 zephyr 系统固件的初始化过程中，用于系统时钟初始化的驱动函数之一 CLOCK_SetPbeMode，其原型如表 3.2 所示，该驱动函数通过一个 while 循环持续读取外设寄存器的值，只有当系统时钟设置完成，外设将设置成功的标志信息反馈到寄存器后， while 循环才会终止，而由于模拟器中并没有实际的外设支持，这个寄存器的值永远不会变化，这就导致了固件样本无法陷入死循环，无法完成初始化工作。通过直接返回零而忽略对固件与外设的交互逻辑的处理就可以很好地解决这类问题。

表 3.2 CLOCK_SetPbeMode 函数代码片段

```
1. status_t CLOCK_SetPbeMode(mcg_pll_clk_select_t pllcs,mcg_pll_config_t
   *config)
2. {
3.     do {
4.     } while ((DAT_40064006 & 0x1c) != 8);
```

```
    return 0;
}
```

2) 基于实现重要初始化逻辑的函数替换方案

对于少部分的初始化函数，仅仅返回一个标志是不够的。这些函数往往存在着对系统正常运行有不能被忽略的关键操作，例如对系统中断的定义，外设定义结构体的操作等。对于这些函数的替换需要手动实现函数中程序运行所必要的关键函数逻辑。

例如初始化网卡驱动的 HAL_ETH_Init 函数，尽管我们并不需要关心该函数的大部分对外设配置的工作，但是 FreeRTOS 操作系统重写了 STM32 为用户提供的自定义初始化接口函数 HAL_ETH_MspInit，并在这个函数中完成了使能网络中断，设置网络中断优先级这两个重要操作。因此，对 HAL_ETH_Init 的替换不能选择直接进行跳过，而是要在 Hook 函数中手动实现网络中断的使能，这个工作包括将网络中断号加入中断控制模型的使能列表中，并将 Unicorn 引擎中对应寄存器置位。

3) 基于模拟 MCU 特性的初始化支持方案

Unicorn 引擎无法完全模拟的 MCU 状态信息，有时也会对 RTOS 系统的初始化工作造成影响。在这种情况下，工具通过编写钩子函数，来完善 Unicorn 引擎对 MCU 特性的模拟能力。

一个典型例子是 Free RTOS 用于开启多任务调度功能的 xPortStartScheduler 函数。在这个函数中存在一个检查 FreeRTOS 的优先级设置与硬件设置是否一致的操作，如果不一致，则触发 configASSERT 机制，使程序陷入死循环。FreeRTOS 的 xPortStartScheduler 函数的优先级位数检查方式是在内存地址为 0xE00E400 的中断优先级寄存器处写入 0xff 再读出。由于 Arm Cortex M 架构特性，这个寄存器的低比特位无法读写，因此读出来的数值中部分低比特位为 0，具体的读出值将取决于 MCU 的实际优先级比特数设置。然而，与真实的 MCU 处理器不同，对于 Unicorn 引擎来说，这个地址与其他内存空间并无区别，因此读出的数据仍是 0xff，这就导致优先级检测失败，程序陷入死循环。

通过 Unicorn 的 HOOK_MEM_WRITE 机制，编写钩子函数，模拟 MCU 对这部分地址的读写控制，可以绕过检查机制，解决这个问题。该函数流程如图 3.4 所示，函数基本思路如下：

1. 计算优先级掩码，这是一个 8 位的二进制数，其中高位是优先级位，低位是 0，

优先级位的个数由 PRIO_BITS 变量决定。

2. 计算优先级值，这个值是原优先级值和优先级掩码进行按位与运算得到结果，优先级值表示了中断的优先级。
3. 使用 uc.mem_write 方法，将转换为字节序列后的优先级值写入到原地址。

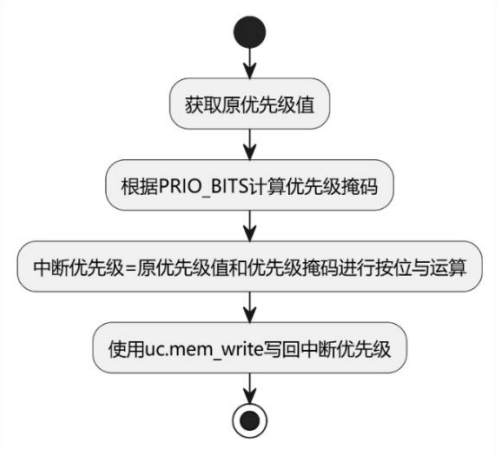

图 3.4 模拟中断优先级机制的 Hook 函数流程图

二、日志记录功能

在尝试进行固件仿真时，拦截并记录固件发出的日志消息，尤其是错误信息是一项不可或缺的一项关键任务。通过这种方式可以检测错误发生的位置、错误的来源，同时常常可以获得人类可读的消息。因此本工具的库函数替换模块提供了捕获日志信息的 Hook 函数，尽量将函数中提供的信息转化为人类可读的字符串。

LiteOS 提供了一个名为 LOS_LkPrint 的函数用于记录日志信息，这个函数可以产生 TRACE_DEBUG，TRACE_COMMOM，TRACE_ERROR 等多个不同 level 的日志信息，函数通过固件中设置的默认日志等级来决定是否生成某个 Level 的日志信息。类似的，FreeRTOS Plus 库提供了一个 UDPLoggingPrintf 函数，这个函数的目的是产生并输出 FreeRTOS 的日志信息到 UDP 端口。有些 RTOS 会产生控制台信息，这些控制台信息大多是以 Uart 协议为传输介质进行发送，并传输到显示器设备中，这些也是便于人类阅读的信息。同时很多操作系统中也常使用 printf 等函数实现标准输出，工具同样提供了针对这些函数的 Hook 函数，以尽可能多地获取运行过程中产生的信息。

上述这些函数虽然来自不同的操作系统，内部实现也不尽相同，但它们都有着相似的功能与相似的参数，例如 LOS_LkPrint 函数原型如表 3.4 所示。针对这些函

数的钩子函数的设计思路也都是类似的，替换这些日志输出函数的钩子函数的算法流程如图 3.5 所示。其函数步骤如下：

1. 获取参数：level，func，line 和 fmt。
   a) level 表示日志的级别
   b) func 表示日志所在的函数名
   c) line 表示日志所在的行号
   d) fmt 表示日志的格式字符串。
2. 根据 level 的值从 level_dict 字典中获取对应的级别字符串，如果没有找到则使用默认值"[UNKNOWN]"。
3. 获取 fmt 字符串。
4. 根据 level，func，line 和 fmt 信息拼接完整的日志字符串，包含级别，行号，函数名和格式字符串 log_string
5. 将 log_string 保存在日志文件中。

表 3.3 LiteOS 的 LOS_LkPrint 函数原型

```
1. void LOS_LkPrint(int level, const char *func, int line, const char *fm
   t, ...);
```

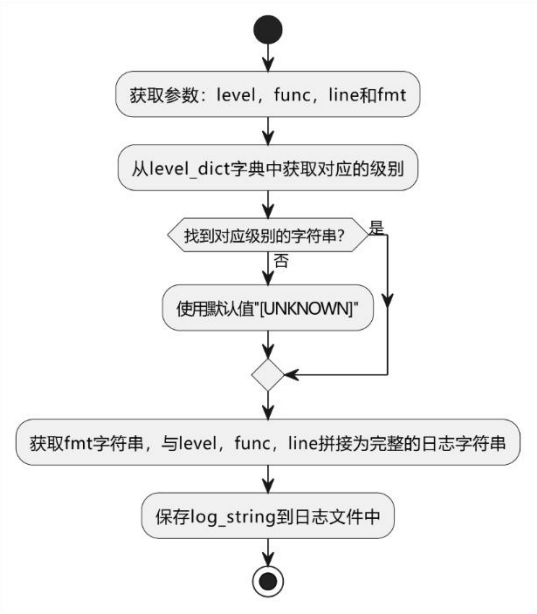

图 3.5 日志记录函数实现流程图

三、中断触发功能

库函数替换模块的中断触发功能，主要负责在中断被触发后，中断回调函数可以将程序引导到正确的执行路径。在中断控制与计时器章节，本文已经介绍了 Arm Cortex M 架构的中断触发机制，在中断触发时，用户设置的中断回调函数会被调用从而对中断进行处理，而外界的信息与数据输入的模拟过程也是由外设通过触发中断，进而触发相应的回调函数完成的。通过我们中断控制器模块，我们可以实现中断的触发并调用相应的回调函数。但是，很多中断请求的回调函数与外设和硬件信息之间有着较为紧密的联系，不加处理的话，程序并不会执行到我们希望的程序路径，这会对我们的固件仿真工作带来问题。

一个例子是 ST 公司的硬件驱动库中用于处理 UART 中断触发的回调函数 HAL_UART_IRQHandler，该函数原型如表 3.4 所示。这个回调函数调用 __HAL_UART_GET_FLAG 函数读取外设寄存器组中对应 flag 标志位，根据标志位是否置位确定是否调用相应的回调函数。由于我们并不维护外设，以及外设寄存器，HAL_UART_IRQHandler 函数永远不会调用 UART_Receive_IT 函数，也就无法实现 UART 串口数据的读取。

为了解决这个问题，工具对无法触发正确路径的中断函数进行了 Hook 替换，修改它们的逻辑，跳过获取外设信息的环节，直接手动调用我们模糊测试过程中需要的，用于接收数据的驱动函数，模拟数据接收行为。对 UART 中断函数进行替换的 Hook 函数如表 3.5 所示，它通过直接将 PC 寄存器的值覆写为 HAL_UART_Receive 的方式调用驱动函数进行数据接收。

表 3.4 HAL_UART_IRQHandler 部分代码

```
1. void HAL_UART_IRQHandler(UART_HandleTypeDef *huart)
2. {
3.   uint32_t tmp1 = 0, tmp2 = 0;
4.   ......
5.   tmp1 = __HAL_UART_GET_FLAG(huart, UART_FLAG_RXNE);
6.   tmp2 = __HAL_UART_GET_IT_SOURCE(huart, UART_IT_RXNE);
7.   /* UART in mode Receiver ----------------------------------------
   --------*/
8.   if((tmp1 != RESET) && (tmp2 != RESET))
9.   {
10.    UART_Receive_IT(huart);
11.  }
12.  ......
13.}
```

表 3.5 HAL_UART_IRQHandler 钩子函数

```
1. def HAL_UART_IRQHandler(uc):
2.     # call the UART callback function by my self
3.     uc.reg_write(
4.         UC_ARM_REG_PC, uc.symbols['HAL_UART_Receive'])
```

四、串口通信功能

作为一种简单、有效、灵活的串行通信协议，UART（universal asynchronous receiver / transmitter 通用异步收发器）在嵌入式系统中具有广泛的应用与意义。为了完善固件模拟功能，对嵌入式固件样本中 UART 功能的实现是必不可少的。

在对 RTOS 固件的网络功能与串口通信功能的重托管工作中，我发现拦截硬件开发商提供的底层驱动函数以及较低级别的回调函数比拦截较高级别的回调容易得多。通过尽量不去改动高级别的回调函数的方式，我们可以方便地利用 RTOS 系统的上层函数中提供的读写函数处理信号量和进程间通信模块。这些信号量和通信机制是在固件中的操作系统上实现调度，执行多个任务所必需的，而让我们自己编写的 Hook 函数操作并管理这些信号量是一项难度极大甚至不可能实现的任务。因此，对于串口通信和网络接口功能的支持工作，库函数替换的关注重点放在了与设备生产商密切相关的硬件驱动函数上。

ST 公司为其开发的 STM32 系列硬件产品提供了接口统一的 UART 驱动程序，在我们选择的 FreeRTOS 固件样本及 LiteOS 固件样本中，这些驱动程序被上层系统调用以完成串口通信任务。Zephyr 操作系统的底层驱动为 UART 提供了一系列统一的接口，它们一般组织为 uart_xxx_poll_in，uart_xxx_poll_out，uart_xxx_fifo_read 的形式，其中 xxx 一般为适配的硬件厂商名。对于这些驱动库函数的替换工作主要关注利用 UART 协议进行串口数据收发的关键函数，这些函数如表 3.6 与表 3.7 所示。这些函数都具有相似的函数功能，函数参数也是类似的。用于替换 UART 数据接收函数的钩子函数只需要将对应字节数的字节数据从 FuzzerIO 模型中获得，并写入函数指针对应位置即可。

表 3.6 zephyr 操作系统用于 UART 协议 IO 操作的驱动函数原型

```
1. int uart_mcux_fifo_read(device *dev,uint8_t *rx_data,int len)
2. int uart_mcux_poll_in(device *dev,uchar *c)
3. void uart_mcux_poll_out(device *dev,uchar c)
```

表 3.7 STM32 系列驱动库中用于 UART 的部分驱动函数原型

```
1. HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t
   *pData, uint16_t Size)
2. HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t
    *pData, uint16_t Size)
```

下面我将简述负责串口接收功能的钩子函数设计思路，构造的钩子函数流程如表 3.10 所示。其函数步骤如下：

1. 从 R1 寄存器读取一个指针 p_char，它指向要接收字符的内存地址。
2. 调用外设模型 UARTModel 的 read 方法，从 FuzzerIO 模块读取一个字符。返回值是一个字节串 data。
3. 如果 data 的长度为 1，说明读取成功，那么打印出接收到的字符，并把 data 写入 p_char 指向的内存地址。然后把 0 写入 R0 寄存器，表示成功状态，并返回。
4. 如果 data 的长度不为 1，说明读取失败，那么把 0xFFFFFFFF 写入 R0 寄存器，表示错误状态，并返回。

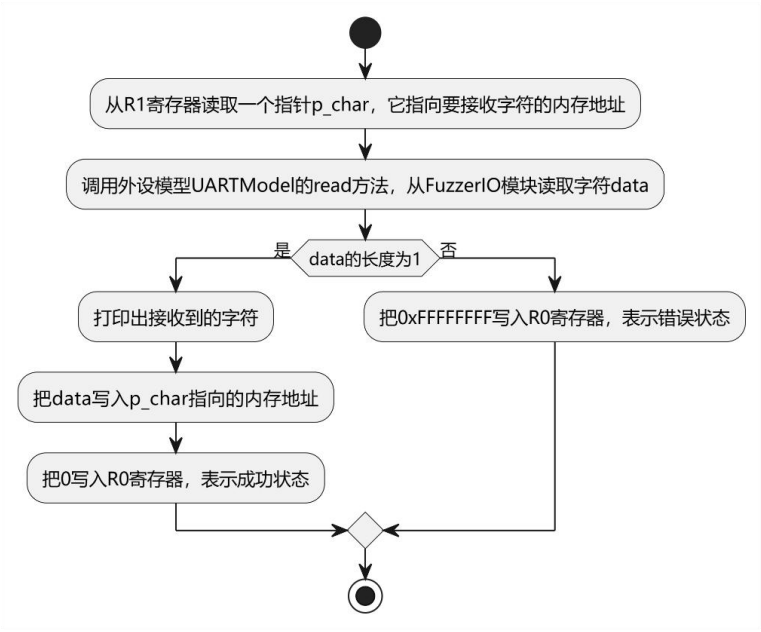

图 3.6 串口接收钩子函数流程图

五、网络通信功能

嵌入式设备的网卡驱动是一种用于实现嵌入式设备与网络通信的软件模块，它

负责将网卡硬件接收到的数据包传递给网络协议栈。针对 RTOS 系统的网络功能的钩子函数设计，主要关注点放在实现网卡驱动与上层网络协议栈交互的接口函数中。这类函数主要包括数据帧的收发函数与网卡 PHY 寄存器读写函数两部分。对于 STM 系列的网络驱动程序，这些函数统一为 HAL_ETH_XX 形式，同样，NXP 也提供了一系列网络驱动程序，一般组织为 ENET_XX 形式。这些网卡驱动函数的函数原型如表 3.8 所示。下面我将分别介绍对数据帧的收发以及网卡 PHY 寄存器读写两个部分函数的钩子函数设计方案。

表 3.8 STM32 部分网卡驱动函数原型

```
1. HAL_StatusTypeDef HAL_ETH_ReadPHYRegister(ETH_HandleTypeDef *heth,uint16_t PHYReg,uint32_t *RegValue)
2. HAL_StatusTypeDef HAL_ETH_WritePHYRegister(ETH_HandleTypeDef *heth,uint16_t PHYReg,uint32_t RegValue)
3. HAL_StatusTypeDef HAL_ETH_GetReceivedFrame(ETH_HandleTypeDef *heth)
4. HAL_StatusTypeDef HAL_ETH_TransmitFrame(ETH_HandleTypeDef *heth,uint32_t FrameLength)
```

1. PHY 寄存器读写

PHY 寄存器是网卡芯片中的一种寄存器，它把自己目前的状态反映到寄存器里面，MAC 通过 SMI 总线不断地读取 PHY 的状态寄存器以得知目前 PHY 的状态。基于库函数替换的固件仿真方案并不会关心底层外设的状态，因此对于大部分利用 PHY 寄存器读写的状态信息例如连接速度、双工的能力等，我们并不需要提供相应支持。鉴于这个原因，设计 Hook 函数过程中会忽略 PHY 寄存器写操作，并忽略大部分 PHY 寄存器的读操作，仅在部分可能影响程序运行的 PHY 寄存器读操作发生时才进行处理。例如在获取连接状态的情况下返回 0x01 使程序认为网卡设备总是“连接成功”的，使程序可以继续运行。

2. 数据帧的收发

模拟网卡驱动的数据帧接收功能的Hook函数主要负责完成Fuzz数据的获取工作，同时它也要负责对驱动层重要结构体如网卡控制描述符等的维护工作。它借助外设模型读取 Fuzzer IO 模块的数据并将其写入 Unicorn 引擎用于存储网络帧而开辟的 buffer 空间中。函数通过上述操作来模拟网络帧的接收工作并完成与模糊测试模块的对接。对于发送网络数据帧的函数，工具的 Hook 函数会读取固件发送的帧信息，使用 dpkt 库对发送的帧进行解析，并将解析后的帧信息记录在日志中，

供分析师查看。下面我将简述数据帧收发模块的钩子函数的设计方案。

图 3.7 以太网帧发送钩子函数流程图

实现以太网帧发送功能的钩子函数流程如图 3.7 所示，其实现逻辑如下：

1. 从 R0 寄存器读取 heth 句柄，它是一个指向以太网控制器结构体的指针。
2. 从 R1 寄存器读取 frame_len，它是要发送的数据帧的长度。
3. 从 heth 句柄中获取 DMA 发送描述符列表的首地址，然后通过该描述符的地址域找到要发送的帧的存储地址信息，从 Unicorn 引擎中读取对应的帧内容。
4. 调用 EthernetModel 模型的 tx_frame 方法，传入 heth 句柄的第一个字段（以太网控制器的 ID）和 contents 变量（数据帧内容），这个方法会用 dpkt 库对数据帧进行解析和处理。
5. 将解析信息输出或记录在日志文件中。
6. 将 R0 寄存器写为 HAL_OK，表示函数执行成功。

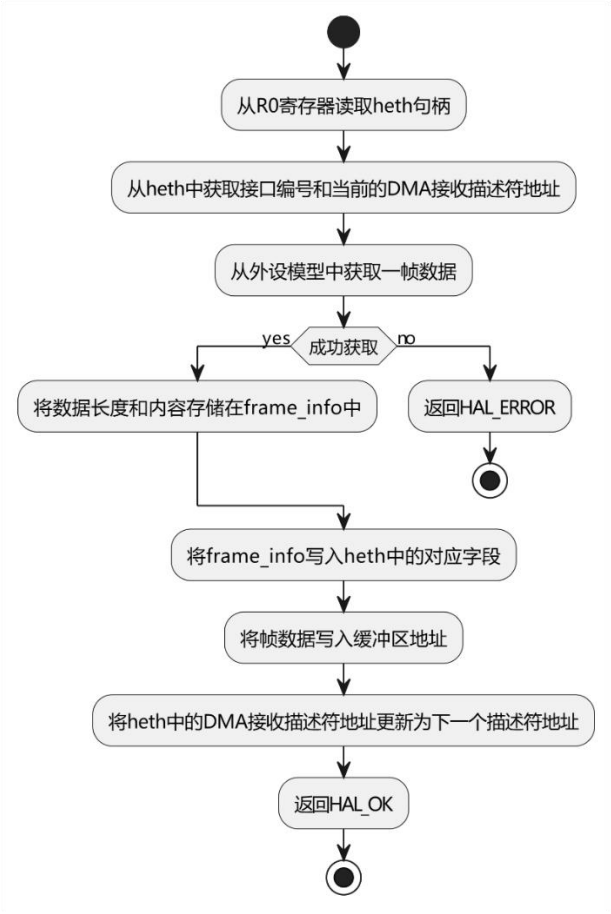

图 3.8 以太网帧接收钩子函数流程图

实现以太网帧接收功能的钩子函数流程如图 3.8 所示，其函数实现逻辑如下：

1. 从 R0 寄存器读取 heth 句柄，它是一个指向以太网控制器结构体的指针。
2. 从 heth 中获取接口的编号和当前的 DMA 接收描述符地址。DMA 接收描述符是一个链表结构，每个节点包含了一个指向缓冲区地址和下一个描述符地址的指针。
3. 从外设模型中获取一帧数据，若成功获取，将其长度和内容存储在结构体 frame_info 中；若未成功获取，返回 HAL_ERROR 指示获取失败。
4. 将 frame_info 写入以太网句柄结构体中的对应字段，表示当前接收到的帧的信息。然后，将帧数据写入缓冲区地址，并将 heth 中的 DMA 接收描述符地址更新为下一个描述符地址。这样就完成了一次帧接收的操作。
5. 返回 HAL_OK，表示操作成功。

六、文件系统功能

库函数替换模块对于文件系统的支持部分的总体方案如图 3.9 所示，工具针对

不同的操作系统灵活选择库函数替换方案，从对底层 IO 的模拟，以及对上层文件系统接口的抽象两个角度进行库函数替换工作。对底层 IO 的模拟，主要是利用 PC 的内存空间来模拟目标机器的存储设备，将对 SD 卡或 Flash 闪存驱动进行 IO 操作替换为对模拟器的内存映射或镜像文件的读写操作。对上层文件系统接口的抽象，主要是使用 POSIX 系统下的等效文件操作函数来代替目标机器的文件系统接口函数，例如用 fopen，fread，fwrite 等函数来实现文件的打开，读取，写入等操作。接下来，我将分别介绍工具对于 FreeRTOS，LiteOS 以及 Zephyr 的文件系统支持工作。

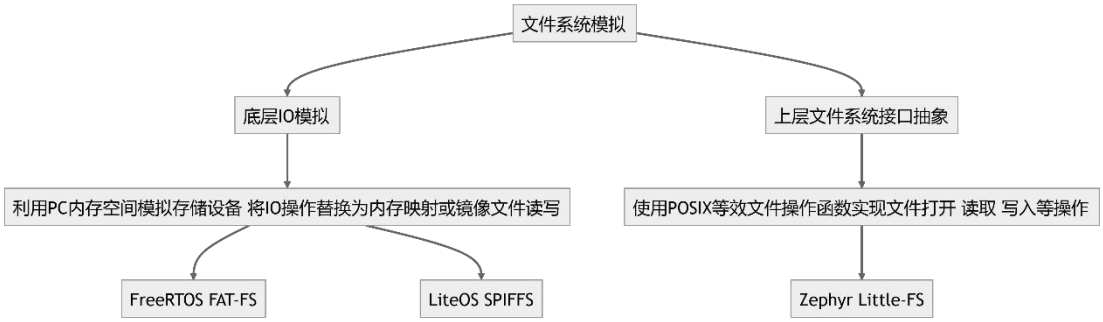

图 3.9 文件系统支持功能实现方案分类

1) FreeRTOS 文件系统

FreeRTOS 提供 FreeRTOS Plus FAT 软件库实现文件系统，其软件栈结构如图 3.10 所示。FreeRTOS 为其文件系统提供了一个通用的文件系统接口 FFI（FreeRTOS File Interface），用于屏蔽不同文件系统实现方式的差异，并提供统一的 API 来操作文件。FFI 所支持的是 FAT（File Allocation Table）格式的文件系统，也就是常见的 FAT16 和 FAT32 格式。

FreeRTOS+FAT 库为了简化与底层驱动的交互工作，在 FFI 层中定义了使用了的名为 FF_BlockDevice_t 的结构体，如表 3.9 所示。这个结构体保存了 FatFS 文件系统与底层 SD 卡或 Flash 闪存驱动进行 IO 操作的关键设备驱动函数指针 fnpWriteBlocks，fnpWriteBlocks，这些函数完成并隐藏了很多底层驱动的操作细节。这两个函数接口在 FF_SDDiskInit 函数中被初始化并绑定。

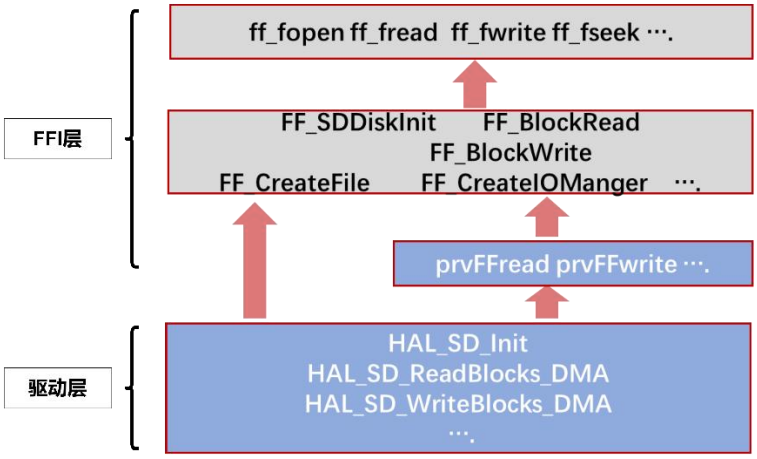

图 3.10 FreeRTOS Plus FAT 软件栈

表 3.9 FF_BlockDevice_t 结构体

```
1. typedef struct
2. {
3.  FF_WriteBlocks_t fnpWriteBlocks; /* Function Pointer, to write a bloc
   k(s) from a block device. */
4.  FF_ReadBlocks_t fnpReadBlocks; /* Function Pointer, to read a block(s)
    from a block device. */
5.  FF_Disk_t *pxDisk;    /* Earlier called 'pParam': pointer to some par
   ameters e.g. for a Low-Level Driver Handle. */
6. } FF_BlockDevice_t;
```

对 FreeRTOS Plus FAT 库的文件系统功能的支持采用抽象并改写程序与磁盘及 SD 卡等外设进行交互的函数的方式进行。工具针对 fnpWriteBlocks，fnpWriteBlocks 这两个函数指针设计了对应的钩子函数进行替换，将调用底层 SD 卡驱动的 IO 操作转化为对自制的 FAT 磁盘镜像文件的读写操作，并将读写内容映射到 Unicorn 引擎的对应的内存中。这个策略成功实现了对 FAT 文件系统的支持。可以看到，在分析过程中通过对 fnpWriteBlocks，fnpWriteBlocks 两个函数的 Hook 操作，解决了棘手的底层驱动 IO 操作的分析问题，这两个 API 接口的存在，大大帮助了高级仿真工作的进行，这也从侧面反映出使用高级仿真技术解决硬件依赖问题在固件分析工作中的巨大潜力。

2) LiteOS 文件系统

LiteOS 的文件系统是一个轻量级的虚拟文件系统，支持多种类型的存储设备，如 Flash、SD 卡、RAM 等。LiteOS 的文件系统主要包括 VFS，ramfs，spiffs 和 fatfs 几个部分。其中，VFS 是其他文件系统的顶层封装，提供了一套通用的文件操作

接口，如 open、read、write、close 等。FATFS 在 FreeRTOS 的文件系统支持过程中已有说明，不再赘述。ramfs 是 LiteOS 中使用的内存文件系统，用于在内存中创建临时文件或目录，由于其不需要与外设的 IO 操作，我们并不需要关注这个文件系统的实现。SPIFFS 是一个专为 SPI NOR flash 闪存设备设计的轻量级的嵌入式文件系统，下面我重点介绍工具对 SPIFFS 文件系统的支持工作。

SPIFFS 系统的支持采用将底层驱动的 IO 操作替换为对模拟器的内存映射读写操作的方案来实现。工具在 Unicorn 引擎中开辟了一个专门用于模拟 NOR flash 闪存设备的内存空间，同时将 LiteOS 为 spiffs 提供的一系列命名为组织为 hal_spi_flash_xxx 的接口函数进行了抽象与替换，将 Flash 闪存设备的 IO 操作转化为对模拟内存空间的 IO 操作。被替换的接口函数如表 3.10 所示。通过库函数的替换工作工具忽略了底层驱动函数使用 SPI 协议的复杂交互过程，使模拟器实现了对 SPIFFS 文件系统的支持。

表 3.10 LiteOS 的 SPI Flash 驱动抽象函数

```
1. void hal_spi_flash_config(void);
2. int hal_spi_flash_erase(uint32_t addr, int32_t len);
3. int hal_spi_flash_write(const void *buf, int32_t len, uint32_t *locati
   on);
4. int hal_spi_flash_erase_write(const void *buf, int32_t len, uint32_t l
   ocation);
5. int hal_spi_flash_read(void *buf, int32_t len, uint32_t location);
6. int hal_spi_flash_get_id(void);
7. void hal_spi_flash_power_down(void);
8. void hal_spi_flash_wake_up(void);
```

3) Zephyr 文件系统

通过对操作系统硬件抽象层或驱动层中用于磁盘，SD 卡等存储设备的初始化工作，状态设置于读取，IO 操作等工作的关键函数进行 hook 与抽象，是使固件可以很好地启动文件系统的一种有效方案，但是我们也注意到，有的操作系统的底层 IO 操作并没有提供简明的接口方便我们进行抽象，同时有时候我们在测试过程中其实并不关心文件系统的底层运行过程，而只要求文件系统可以完成预期的行为，或者我们想要直观地观察到操作系统生成的文件内容，以便更好地对程序进行分析。在上述这些情况下，更好的重托管解决方案是对文件系统的上层 API 进行抽象和模拟。

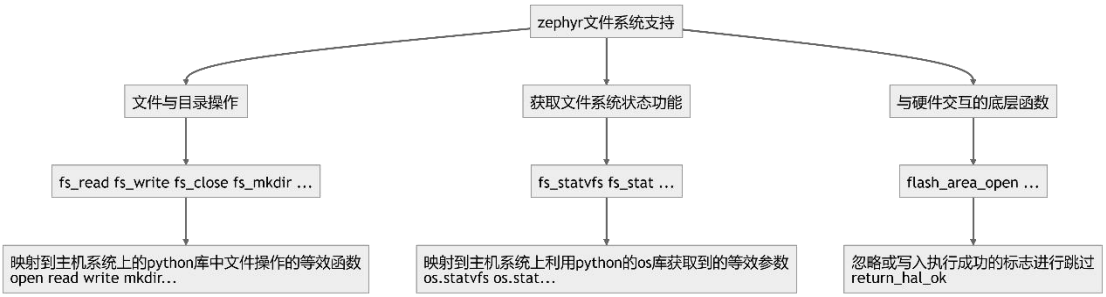

图 3.11 zephyr 文件系统支持策略

工具的库函数替换模块在针对 zephyr 的文件系统的支持中，选择对文件系统的顶层 API 进行抽象，而忽略大部分涉及硬件交互与 IO 的底层的实现细节。具体的实现方案如图 3.11 所示。

对于实现文件与目录操作的 fs_read、fs_write、fs_close、fs_mkdir 等函数工具选择将其直接映射到主机系统上的 python 库中文件操作的等效函数如 read，write 等，将对固件中文件和目录的处理转化为对主机文件和目录的处理。对于获取文件系统状态功能则选择将 zephyr 文件系统特定的参数映射到主机系统上利用 python 的 os 库获取到的等效参数。例如对于获取文件系统挂载状态的函数 fs_statvfs，以及获取文件或目录状态的函数 fs_stat，我们选择调用 Python 库中的 os.stat，os.statvfs 函数获取我们在主机系统对应文件或目录的状态信息，对这些信息进行处理后写回到 Unicorn 引擎对应内存中。而对于与硬件交互的底层函数如 flash_area_open 等则选择简单地忽略或在返回值中写入“执行成功”的标志进行跳过。

3.4 FuzzerIO 模块

FuzzerIO 模块负责加载 AFL 模糊测试工具输入的突变数据并将其分发给库函数替换模块的各外设模型中。这些功能由 load_fuzz 和 get_fuzz 两个函数来完成。load_fuzz 负责在程序开始时将模糊测试的输入数据加载到内存中。get_fuzz 则是提供给库函数替换模块的各外设模型的接口，外设模型通过调用 get_fuzz 函数，从内存中读取任意字节数的模糊测试输入。下面我将简要介绍这两个函数的实现。

1. load_fuzz 函数

load_fuzz 函数由固件模拟与中断控制模块在正式固件模拟前调用，它被用来加载 fuzz 文件，实现对 AFL 提供的突变数据的加载工作。函数执行流程如图 3.12

所示，其主要逻辑如下：

1. 接受 path 参数，表示 fuzz 文件的路径。
2. 打开 fuzz 文件，如果失败则返回错误码。
3. 获取 fuzz 文件的大小，如果失败则返回错误码。
4. 如果 fuzz 文件的大小大于预分配的缓冲区大小（PREALLOCED_FUZZ_BUF_SIZE），则释放原来的缓冲区，并重新分配一个大小为 fuzz_size 的缓冲区，用于存储 fuzz 数据。如果分配失败则返回错误码。
5. 将 fuzz 文件的内容读取到 fuzz 缓冲区中，如果失败则返回错误码。
6. 返回 0 表示加载成功。

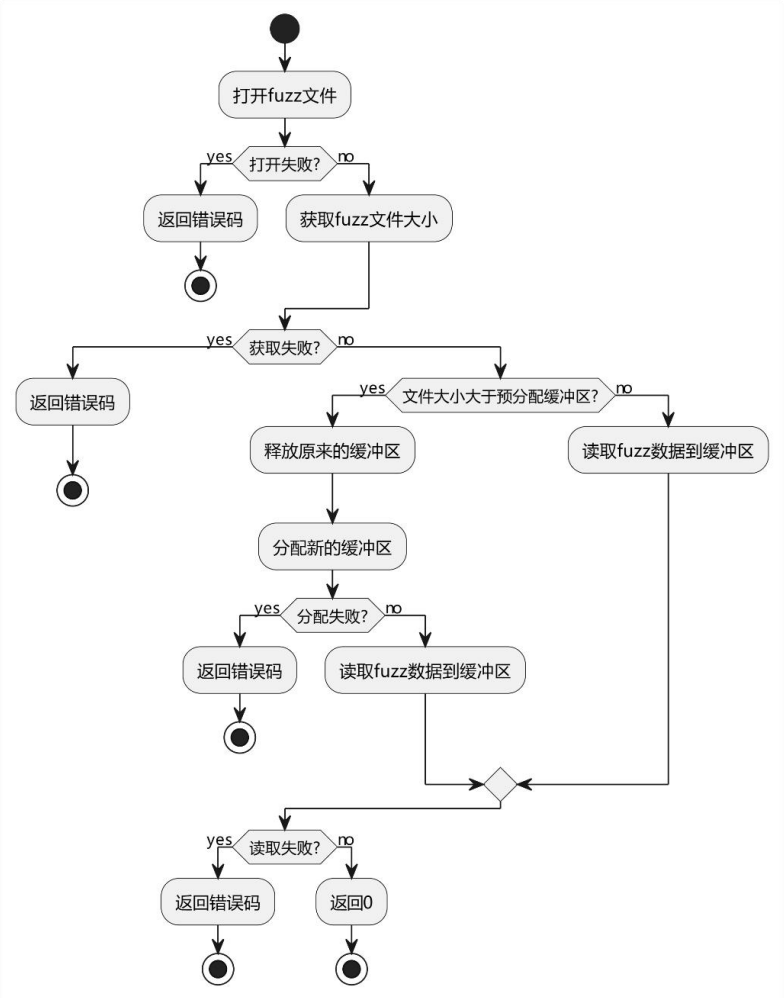

图 3.12 load_fizz 函数实现流程图

2. get_fuzz 函数

get_fuzz 函数实现了为外设模型提供数据的重要功能，它从 fuzz 数据中获取指

定大小的字节序列，并返回一个指向这些字节的指针。函数的执行流程如图 3.13 所示，函数的主要逻辑如下：

1. 首先，检查 size 是否为非零值，以及 fuzz_cursor（表示当前 fuzz 数据的位置）加上 size 是否超过了 fuzz_size（表示 fuzz 数据的总大小）。如果这两个条件都满足，说明还有足够的 fuzz 数据可以获取。
2. 使用 fuzz 和 fuzz_cursor 定义一个 res 变量，作为返回值。这个变量是一个指向 fuzz 数据中 fuzz_cursor 位置的指针。
3. 将 fuzz_cursor 向后移动 size 个字节，以便下次调用该函数时可以获取不同的 fuzz 数据。
4. 返回 res 变量。之后外设模型或取到 res 指针后，从这个指针开始，读取 size 个字节。

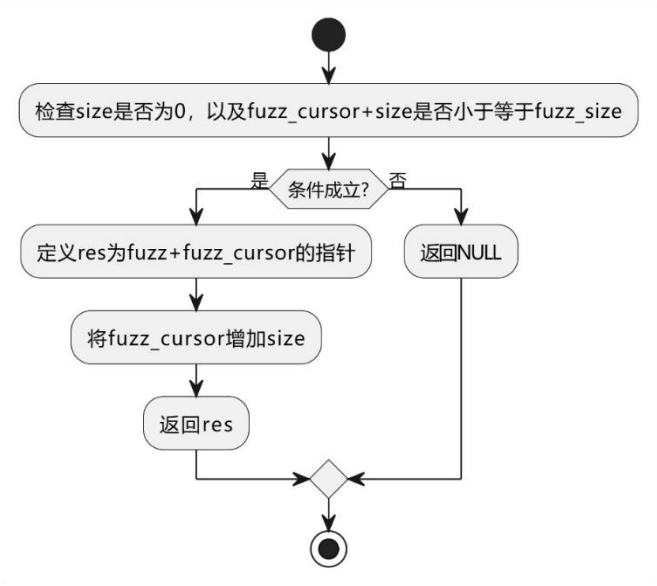

图 3.13 get_fuzz 函数实现流程图

库函数替换模块通过对 load_fuzz 和 get_fuzz 两个函数接口的应用，实现了与模糊测试工具 AFL 的对接，使工具具备了对固件样本进行模糊测试的能力。

3.5 本章小结

本章介绍了基于固件仿真的物联网操作系统模糊测试工具的系统设计及功能实现。系统架构主要由 Fuzzer IO 模块，库函数替换模块，固件模拟与中断控制模

块三部分组成。工具针对操作系统的初始化工作，日志记录，网络与串口通信，文件系统等模块的关键函数做了库函数替换工作，解决了硬件依赖问题，实现了嵌入式固件的模拟仿真，模糊测试流程，可以对固件进行模糊测试并输出结果。

第四章 测试

4.1 测试案例与测试环境

4.1.1 测试案例选择

1) 针对 FreeRTOS 的测试案例

对于 FreeRTOS 操作系统，测试案例选择了运行在和 FreeRTOS 有着密切合作的 ST 公司推出的 STM32 系列微控制器上的固件样本。FreeRTOS 官网提供了一个基于 STM32 的示例 Demo 样例，这个样例开启了一个网络服务器，用户可以向其发送 HTTP 请求并获取回复。该案例涵盖了 FreeRTOS 的 TCP，UDP 等网络协议软件栈的调用，基于 Free RTOS Plus FAT 库的文件系统实现，以及控制台和 Uart 串口协议。以这个固件 Demo 为案例可以很好地涵盖我们的模糊测试工具针对 FreeRTOS 操作系统各个模块进行评估的需求。

2) 针对 LiteOS 的测试案例

LiteOS 操作系统测试案例来源于 Huawei 的 OpenHarmony kernel-liteos-m 项目仓库，测试过程需要能够完整评估工具对 Zephyr 操作系统的三个主要外设模块，即串口通信，网络传输和文件系统的模拟能力，为此我选择了 LiteOS_SensorHub，LiteOS_spiffs，LiteOS_dhcpv6 三个 demo 来进行测试。LiteOS_spiffs 为 LiteOS 提供的 SPIFFS 文件系统使用案例，该固件完成了 SPIFFS 文件的挂载，示例文件的创建操作。LiteOS_SensorHub 为 LiteOS 的传感器使用案例，其利用 I2C 及 Uart 串口传输协议，收集传感器陀螺仪数据并输出到控制台。LiteOS_dhcpv6 示例为 LiteOS 使用 dhcpv6 协议的一个示例，它完成了通过与 DHCP 服务器协商，获取本机的 IP 地址的工作。三个样例均运行在野火 Cloud_STM32F4 系列开发板上。

3) 针对 Zephyr 的测试案例

Zephyr 操作系统的测试案例来源于 zephyr 官方 demo 库，这个 demo 库涵盖 zephyr 的各个模块与各个功能。为了完整评估工具对文件系统，网络功能，串口通信三个方面的模拟效果，我针对这三个方面分别选取了 zephyr_little_fs，zephyr_dhcpv4，zephyr_uart 三个案例程序进行测试。zephyr_little_fs 依次完成了文

件系统的挂载，目录创建，文件创建，删除等一系列重要操作，这使我们可以方便地测试 zephyr 的文件系统功能。zephyr_dhcpv4 说 zephyr 使用 DHCP 协议与服务器协商本机 IP 地址的示例。Zephyr_uart 是 zephyr 用来演示 UART 串口通信的案例，它简单地从串口收集字符信息并将其再通过 UART 协议回传到控制台上。这些 demo 运行在 NXP 推出的 Frdm_K64f 开发板以及 ST 推出的 STM32 系列开发板上。

4.1.2 测试环境

表 4.1 测试环境配置表

测试环境	版本
UBUNTU	18.0.4
PYTHON	3.7.16
UNICORN	1.0.3
AMERICAN FUZZY LOP	2.52b
CPU	Intel(R)Core(TM)i7-10875H

4.2 模拟仿真功能测试

4.2.1 测试方案

本小节对工具的模拟仿真功能进行测试。模拟仿真功能的效果如何，主要取决于模拟的准确度与替换后的功能一致性两个方面。基于以上考量，本节对工具模拟仿真功能的评估过程主要关注以下几个重点。

- 完成高级替换后固件是否能够执行到正确的程序路径。
- 完成高级替换后固件是否能够获取到来自外部输入的数据。
- 完成高级替换后固件是否产生合理的日志或输出。

本节通过使用工具对选取的固件案例进行模拟执行，记录库函数替换模块中各关键 Hook 函数被调用的次数，并对执行过程中的执行路径，输入输出和产生的日志文件等进行分析与评估，来确定程序的运行逻辑是否符合预期，从而评估工具的模拟仿真功能的实现效果。工具的测试过程全部在 Ubuntu18.0.4 操作系统，

Intel(R)Core(TM)i7-10875H 处理器上进行。为了正确评估工具的模拟仿真功能的效果，本节针对工具对固件的初始化，日志系统，网络通信，串口通信，文件系统等各个功能的支持，提出了各自对应的评估方案，这些方案的设计与测试的结果如 4.2.2 节所示。

4.2.2 测试结果

1) **初始化**

评估固件初始化是否正确完成的标准是程序是否成功启动并调度到用户定义的第一个线程任务中。例如，对 FreeRTOS 操作系统来说，开发者编写程序时一般会倾向于在程序进入 main 函数后，执行 HAL_Init 等初始化函数，写 GPIO 引脚，初始化 TCP/IP 协议栈，创建用户定义的 Task 等工作，最后调用 vTaskStartScheduler 函数开启任务调度，转到用户编写的 Task 线程中。对于本文使用的固件案例 FreeRTOS_TCP_FAT_CLI_Demo 来说，这个 Task 为 prvServerWorkTask 函数。通过分析固件运行的函数调用记录，观察函数是否调用 vTaskStartScheduler 并正确进入用户定义的 prvServerWorkTask 任务中，就可以判断固件是否正确完成了初始化工作。

通过分析与评估，本文涉及的各 RTOS 操作系统固件样本均可正常完成初始化工作。

2) **日志系统**

评估程序是否正确调用日志系统的方法是检查日志文件，观察对应日志记录函数被调用后，程序是否 Hook 到函数并产生正确的 LOG 消息并输出到本机日志文件中。通过对日志记录函数的高级替换捕获到的部分日志信息如图 4.1 所示。

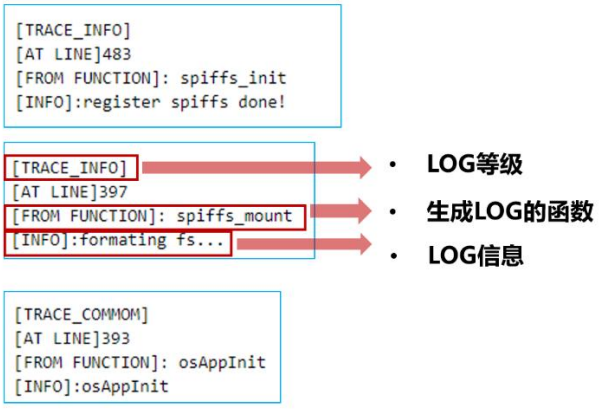

图 4.1 通过 LOS_LkPrint 函数获取的部分日志信息

在测试中，我分别对 Zephyr，LiteOS，FreeRTOS 三个系统的日志拦截函数的调用次数与日志记录能力等内容进行了统计，结果如表 4.2 所示，可以看到工具已基本实现了对三个 RTOS 系统的日志记录功能。

表 4.2 日志拦截函数相关统计表

RTOS 名称	函数名称	调用次数	是否正常工作
FREERTOS	**lUDPloggingPrintf**	**74**	**是**
LITEOS	**LOS_LkPrint**	**6**	**是**
ZEPHYR	**UartPuts**	**2**	**是**

3) **串口通信**

串口通信是固件与显示器，传感器等外设等进行交互的重要功能。评估程序是否正确调用 UART 软件栈的方法是检查日志信息，以及观察程序输出，确认程序是否正确使用 UART 功能。

例如，对于 Zephyr 系统 UART 测试的固件 demo：zephyr_frdm_k64f_UART，它实现了一个简单的 UART 串口收发操作。该固件调用 console_get_line 函数通过 UART 协议接收字符串，之后固件调用 z_vprintk 函数，再将其通过 UART 协议发送出去，我们只需要检查其负责 UART 接收与发送的回调函数是否正常运行且产生相同的输入输出，就可以知道固件的串口通信功能是否正确执行。对该 demo 输出内容的分析如图 4.2 所示。可以看到利用本文的模拟仿真工具该 demo 可以正常工作。

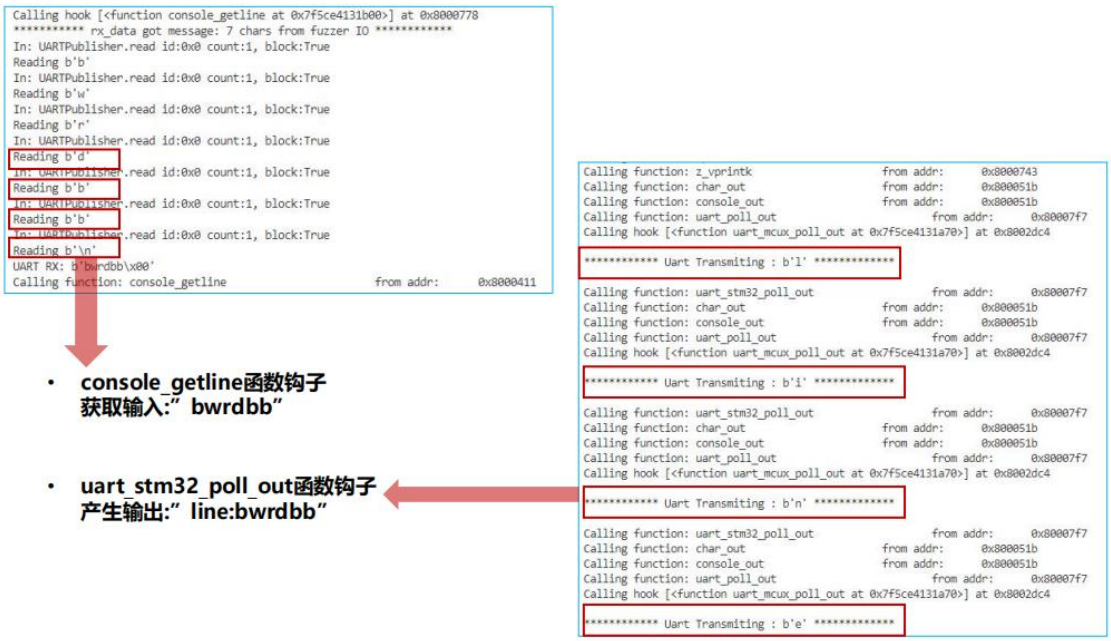

图 4.2 UART 串口功能运行分析示意

在测试过程中，我分别在一次固件模拟过程中，串口接收与串口发送两个关键函数的调用次数进行了统计，并评估了利用 Hook 函数后固件是否能够正确完成预期功能。结果如表 4.3 所示。

表 4.3 串口功能函数相关统计表

函数名称	函数功能	调用次数	是否正常工作
HAL_UART_RECEIVE	**Recv**	8	是
HAL_UART_TRANSMIT	**Send**	145	是
UART_MCUX_POLL_IN	**Recv**	385	是
UART_MCUX_POLL_OUT	**Send**	461	是
UART_MCUX_FIFO_READ	**Recv**	13	是
UARTPUTS	**Send**	2	是
CONSOLE_GET_LINE	**Recv**	10	是

4) **网络通信**

网络通信是固件与外界进行通信与交互的重要功能。网络通信通常涉及复杂的网络协议栈调度，因此对于网络通信功能的评估仅仅观察数据帧的收发函数是否可以正常工作是不够的，我们需要分析接收到的数据包是否可以成功被传递到上层协议栈中并唤醒相应的 Task 或触发相应函数进行响应与处理。

以 FreeRTOS Plus TCP 库的网络协议栈为例，分析过程如图 4.3 所示。

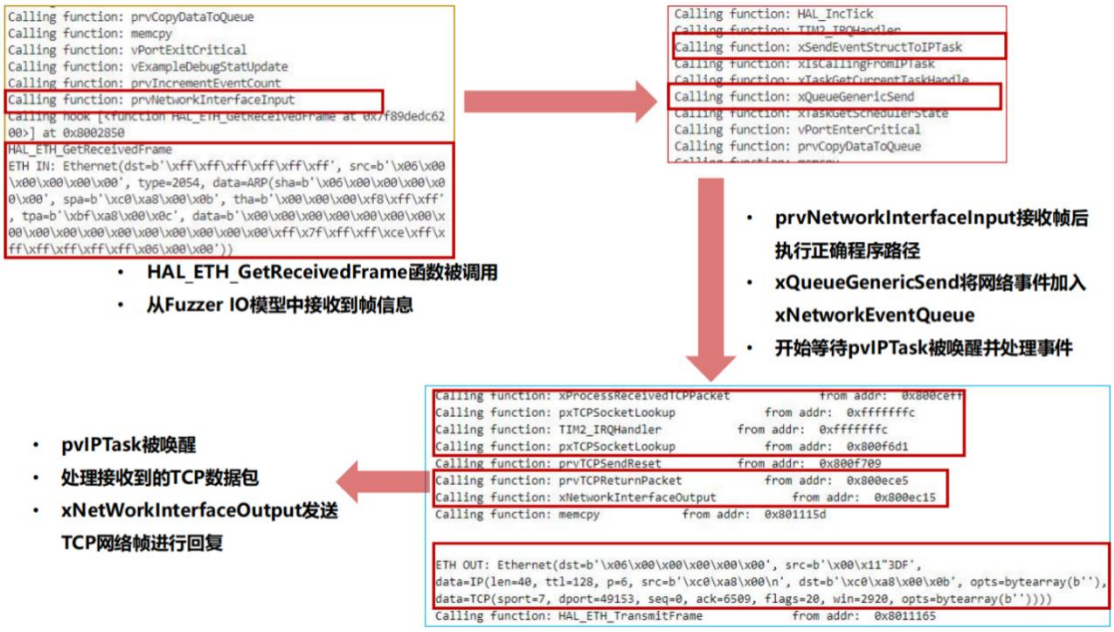

图 4.3 FreeRTOS 网络协议栈调用分析

当发生网络中断时，中断服务例程唤醒 prvEMACHandlerTask 任务，该任务调用 prvNetworkInterfaceInput 函数进行数据帧的接收，prvNetworkInterfaceInput 函数的底层利用驱动函数 HAL_ETH_GetReceivedFrame 进行数据帧接收，并在接收之后将其放置到 NetworkBuffer 中。我们通过对这个函数的 Hook 与替换实现数据帧的接收，如果替换后的该函数能够按我们的预期进行工作，固件接下来会调用 xSendEventToIPTask 函数，该函数创建网络事件结构体，并使用 xQueueGenericSend 函数将其加入队列中。xTaskNotify 函数唤醒 prvIPTask，prvIPTask 被唤醒后读取 xNetworkEventQueue 队列，获取网络事件并根据网络事件类型交给相应的网络协议栈进行处理。

我分别在一次固件模拟过程中，网络帧接收与发送，PHY 寄存器读取的关键函数的调用次数进行了统计，并通过程序日志分析评估了利用 Hook 函数后固件是否能够正确完成预期功能。结果如表 4.4 所示。

表 4.4 网络功能相关函数统计表

函数名称	函数功能	调用次数	是否正常工作
HAL_ETH_GETRECEIVEDFRAME	RECV	60	是
HAL_ETH_TRANSMITFRAME	SEND	9	是
HAL_ETH_READPHYREGISTER	PHY READ	2	是
HAL_ETH_WRITEPHYREGISTER	PHY WRITE	3	是
ENET_SENDFRAME	RECV	2	是
ENET_READFRAME	SEND	30	是
PHY_GETLINKSTATUS	PHY READ	1	是

5) 文件系统

在固件中使用文件系统时，一般会有挂载，目录与测试文件的生成与删除，等一系列文件系统初始化操作。如果这些操作均成功执行，并没有出现错误信息，则认为文件系统可以按预期进行工作。幸运的是，这些工作的进行通常伴随着便于我们阅读的程序日志生成，这大大方便了我们的测试与评估工作。

例如对 FreeRTOS 系统固件模拟输出的日志信息分析如图 4.4 所示。通过观察固件生成的 LOG 信息提示，可以发现我们的重托管固件正确地在模拟平台中完成了文件系统的挂载，初始化，创建示例文件等行为。

图 4.4 FreeRTOS 文件系统 Log 日志分析

我对各 RTOS 系统固件样本中涉及的各文件系统功能进行了测试与评估，结

果如下图所示，可以看到这些文件系统均可以正常工作。

表 4.5 文件系统功能支持评估表

RTOS 名称	文件系统	存储介质	是否正常工作
FREERTOS	**FAT FS**	SD card	是
LITEOS	**SPIF FS**	Flash	是
ZEPHYR	**LITTLE FS**	Flash	是

4.3 模糊测试功能测试

4.3.1 测试方案

对于模糊测试功能的测试主要聚焦在网络传输，串口通信这两类利用外设完成数据接收与发送功能的固件 demo 中。测试过程中我选择了基本块命中能力与 Path 发现能力两个方面来衡量本文工具的模糊测试性能。

1) 基本块命中能力

基本块命中能力使用工具的基本块命中数来衡量。基本块命中数即在固件执行过程中命中的程序基本块数量，它常被用来衡量 fuzzing 过程中代码覆盖率的情况，基本块命中数越高，则工具模糊测试的代码覆盖率也越高，也就意味着模糊测试工具的性能越好。我在模糊测试过程中统计命中的基本块个数，并将本文工具的基本块命中数与同样利用高级替换方案的固件仿真与模糊测试工具 hal-fuzz ，以及全系统仿真器 QEMU 进行对比，以评估本工具对复杂 RTOS 操作系统进行模糊测试的能力。

2) 新 Path 发现能力

新 Path 发现能力使用在模糊测试工具运行过程中发现的 Path 总数来衡量。AFL 使用了基于覆盖率的 fuzzing 技术，它会记录每个基本块的执行次数，当发现新的基本块时便判断其发现了新的路径。当 AFL 发现新的路径时，它会将该路径添加到队列中，以便在后续的 fuzzing 过程中进行探索。AFL 的 path 发现数量通常是衡量 fuzzing 效果的重要指标，发现新路径的数量越多，就说明 fuzzing 的效果越好。我统计了各固件 fuzz ing 过程中的 path 发现数量，crash 发现数量等内容，以评估工具对复杂 RTOS 操作系统进行模糊测试的能力。

4.3.2 测试结果

1) 基本块命中能力

本文工具以及其他模糊测试工具或模拟器在固件执行过程覆盖基本块个数统计结果如表 4.6 所示。

表 4.6 不同工具固件执行过程覆盖基本块个数统计

固件名称	本文工具	HAL-FUZZ	QEMU
FREERTOS_TCP_FAT_CLI_DEMO	2997	1213	407
ZEPHYR_FRDM_K64F_UART	431	233	48
ZEPHYR_FRDM_DHCPV4	1137	478	101
ZEPHYR_FS	1635	490	76
LITEOS_SENSORHUB	701	213	40
LITEOS_DHCPV6	825	433	54
LITEOS_SPIFFS	1021	325	43
平均值	1246	436	99

可以发现，在全部固件案例中，本文的工具的测试结果均明显优于其他两个工具。本文的工具的测试结果均明显优于其他两个工具 HAL-FUZZ 工具虽然也采用了高级替换的方式进行仿真，但是其缺乏很多针对 RTOS 固件托管的分析与优化工作，同时该工具的计时器与中断机制也存在很多缺陷，因此无法很好适配 RTOS 固件，基本块覆盖率相对低了很多，无法完成固件基本功能的模拟。QEMU 的测试结果普遍是最差的，这是由于 QEMU 中缺少对应的外设支持，导致固件在初始化阶段就会早早卡住，退出或进入死循环。

本文工具在测试固件的基本块覆盖数平均为 HAL-FUZZ 工具的 2.85 倍，为全系统仿真器 QEMU 的基本块覆盖数的 12.57 倍。可以看到，本文的工具的测试结果均明显优于其他两个工具，可以认为本文工具的模糊测试能力优于其他主流的嵌入式系统模糊测试工具。

2) 新 Path 发现能力

在测试过程中，我统计了对固件的串口通信和网络通信功能的固件样本进行的模糊测试工作的运行时间，执行总数，新 Path 发现数，crash 发现数等内容。测试

结果如表 4.7 所示。

表 4.7 模糊测试实验结果

固件名称	运行时间	执行总数	PATH 发现	CRASH
FREERTOS_TCP_FAT_CLI_DEMO	0d:7h	63k	447	0
ZEPHYR_FRDM_K64F_UART	0d:18h	570k	210	68
ZEPHYR_FRDM_DHCPV4	0d:10h	119k	389	0
LITEOS_SENSORHUB	0d:6h	284k	40	5
LITEOS_DHCPV6	0d:4h	85k	201	0

在测试过程中可以看到模糊测试工具保持着一定的 path 数量增长速度，并具有一定的 crash 发现能力，可以很好地完成模糊测试工作。综上，可以认为本文的模糊测试工具有较强的模糊测试能力。

4.4 本章小结

本章介绍了基于固件仿真的模糊测试工具的实验评估工作。本章首先介绍了测试用例的选取以及测试环境的选择，之后具体介绍了工具的测试与评估工作。评估工作从模拟仿真功能和模糊测试能力两个方面进行。对模拟仿真功能的测试对工具的日志系统，网络与串口通信，文件系统模拟能力进行了评估。对模糊测试功能的测试评估了基本块覆盖能力和路径发现能力，并与其他当前主流的基于库函数替换的工作做了对比。实验评估结果表明本文所实现的模糊测试工具具有更强的固件仿真及模糊测试能力。

第五章 总结与展望

5.1 论文总结

本文在现有固件仿真与模糊测试研究的基础上，介绍了一种基于固件仿真的嵌入式设备模糊测试方法，该方法能够有效地分析和测试嵌入式操作系统的安全漏洞。本文改进了现有的固件仿真技术，采用了对嵌入式操作系统的硬件驱动和内部库进行分析和替换的方法，解决固件仿真的硬件依赖问题，提高了固件仿真的准确性和适配性。本文设计并实现了一个针对复杂物联网操作系统的模糊测试工具，该工具能够实现对嵌入式操作系统的固件仿真工作，并集成 AFL 模糊测试工具对固件进行模糊测试。本文通过实验验证了该方法和工具的有效性和实用性，为物联网和嵌入式操作系统的安全保障提供了一种新的技术手段。

5.2 未来工作展望

通过本文的研究已初步实现了针对物联网操作系统的固件仿真与模糊测试工具，较好地填补了目前嵌入式设备固件仿真领域的空缺，但仍旧存在一定的提升空间。

（1） 本文针对部分 RTOS 操作系统的部分重要功能进行分析与替换工作，在未来还有更多复杂嵌入式操作系统的库函数的替换工作需要实现。

（2） 目前的库函数替换工作主要聚焦在驱动层以及较为底层的系统库函数，虽然利用固件开发商提供的驱动函数很多时候可以解决硬件依赖问题，但这也就意味着工具要针对每个不同的硬件开发商完成不同的驱动层库函数替换工作，而如果选择仅针对 RTOS 中间件库的底层函数进行替换，那么我们就要在设计的钩子函数中手动解决系统复杂的信号量交互，进程调度等问题，如何权衡替换难度与可重用度二者的关系，提出更好的库函数替换方案也是进一步工作与研究的重点。

（3） 针对模糊测试功能，当前的 crash 检测主要依赖程序本身产生错误信号，而很多错误如内存溢出等问题并不会产生明显异常，也就不容易被模糊测试工具捕获，如何优化模糊测试工具的 crash 检测机制也需要进一步研究。

致 谢

随着毕设工作的告一段落，预示着四年的大学生活即将结束。我很庆幸在大学 时光里遇到如此多的有趣的人和事。在此我向所有给予我帮助、 指导、鼓励、一起前行的人们表示诚挚的感谢，正是因为有你们在，才成就了今天的我。在完成本次毕业设计的过程中，我得到了许多人的帮助和支持，特此在此表示衷心的感谢。

首先，我要感谢我的指导老师傅晓彤教授，以及为我毕设工作提供很多帮助的张宁老师，她们在选题、设计、实验、论文撰写等方面给予了我很多宝贵的指导和建议，让我受益匪浅。张宁老师不仅教会了我专业知识和技能，还培养了我科学的思维方法和严谨的学术态度，对我的成长和发展有着重要的影响。

其次，我要感谢实验室里的同学们。他们在我毕设过程中给予了我很多帮助，在我遇到困难时细心为我答疑解惑，帮助我解决问题，有了他们的帮助我才能克服众多的困难和挑战。

再次我要感谢我可爱的室友们，他们是我在大学生活中最亲密的伙伴，也是我最珍惜的朋友。他们不仅给了我很多有用的建议和意见，还在我遇到困难时给予了我鼓励和安慰。我要向他们表达我最诚挚的感谢和敬意。他们的友谊是我一生的财富，我将永远铭记在心。

再次，我要感谢我的家人，他们一直是我最坚强的后盾，给予了我无限的爱和支持，他们对我的爱和信任让我有了无限的勇气和动力。他们为我的学习和生活提供了良好的条件和环境，他们对我的期望和信任激励着我不断进步和努力，他们是我最珍贵的财富。

最后，我要感谢学校和学院，他们为我提供了良好的学习环境和条件，让我在这里度过了四年难忘的大学时光。在这里，我不仅学到了知识，还拓展了视野，增长了见识，成长了人格。

在此，我对所有关心和帮助过我的人表示最诚挚的感谢！

参考文献

[1] Huawei LiteOS[EB/OL]. https://www.huawei.com/minisite/liteos/cn/.

[2] The FreeRTOS Kernel[EB/OL]. https://www.freertos.org/.

[3] Zephyr 项目 - Zephyr 项目[EB/OL]. https://zephyrproject.org/.

[4] SERPANOS D N, VOYIATZIS A G. Security challenges in embedded systems[J]. ACM Transactions on embedded computing systems (TECS), 2013, 12(1s): 1-10.

[5] KOLIAS C, KAMBOURAKIS G, STAVROU A, 等. DDoS in the IoT: Mirai and other botnets[J]. Computer, 2017, 50(7): 80-84.

[6] MCFAIL M, HANNA J, REBORI-CARRETERO D. Detection Engineering in Industrial Control Systems. Ukraine 2016 Attack: Sandworm Team and Industroyer Case Study[R]. MITRE CORP MCLEAN VA, 2022.

[7] NADIR I, MAHMOOD H, ASADULLAH G. A taxonomy of IoT firmware security and principal firmware analysis techniques[J]. International Journal of Critical Infrastructure Protection, 2022: 100552.

[8] Chen D D, Woo M, Brumley D, et al. Towards automated dynamic analysis for linux-based embedded firmware[C]//NDSS. 2016, 1: 1.1-8.1.

[9] SCHARNOWSKI T, BARS N, SCHLOEGEL M, 等. Fuzzware: Using Precise {MMIO} Modeling for Effective Firmware Fuzzing[C]//31st USENIX Security Symposium (USENIX Security 22). 2022: 1239-1256.

[10] 于颖超, 陈左宁, 甘水滔, 等. 嵌入式设备固件安全分析技术研究[J]. 计算机学报. 2021,5,44. 859-887.

[11] FreeRTOS - 适用于具有物联网扩展功能的嵌入式系统的市场领先 RTOS（实时操作系统）[EB/OL]. https://freertos.org.

[12] 郑尧文, 文辉, 程凯, 等. 物联网设备漏洞挖掘技术研究综述[J]. 信息安全学报, 20191018, 4(5): 61-75.

[13] Zaddach J, Bruno L, Francillon A, et al. AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares[C]//NDSS. 2014, 23: 1-16.

[14] Muench M, Nisi D, Francillon A, et al. Avatar 2: A multi-target orchestration platform[C]//Proc. Workshop Binary Anal. Res.(Colocated NDSS Symp.). 2018, 18: 1-11.

[15] GUSTAFSON E, MUENCH M, SPENSKY C, et al. Toward the analysis of embedded firmware through automated re-hosting[C]//22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019). 2019: 135-150.

[16] FENG B, MERA A, LU L. P2im: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling[C]//Proceedings of the 29th USENIX Conference on Security Symposium. 2020: 1237-1254.

[17] CAO C, GUAN L, MING J, et al. Device-agnostic firmware execution is possible: A concolic execution approach for peripheral emulation[C]//Annual Computer Security Applications Conference. 2020: 746-759.

[18] Zhou W, Guan L, Liu P, et al. Automatic Firmware Emulation through Invalidity-guided Knowledge Inference[C]//USENIX Security Symposium. 2021: 2007-2024.

[19] Chen D D, Woo M, Brumley D, et al. Towards automated dynamic analysis for linux-based embedded firmware[C]//NDSS. 2016, 1: 1.1-8.1.

[20] Li W, Guan L, Lin J, et al. From Library Portability to Para-rehosting: Natively Executing Open-source Microcontroller OSs on Commodity Hardware[C]//NDSS 2021. 2021.

[21] CLEMENTS A, GUSTAFSON E, SCHARNOWSKI T, et al. Halucinator: Firmware re-hosting through abstraction layer emulation[C]//Proceedings of the 29th USENIX Security Symposium. 2020.

[22] YUN J, RUSTAMOV F, KIM J, et al. Fuzzing of Embedded Systems: A Survey[J]. ACM Computing Surveys, 2022, 55(7): 1-33.

[23] CHEN J, DIAO W, ZHAO Q, et al. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing[C]. // Network and Distributed System Security Symposium. 2018.

[24] CLEMENTS A A, GUSTAFSON E, SCHARNOWSKI T, 等. HALucinator:

firmware re-hosting through abstraction layer emulation[C]//Proceedings of the 29th USENIX Conference on Security Symposium. USA: USENIX Association, 2020: 1201-1218

[25] Shoshitaishvili Y, Wang R, Hauser C, et al. Firmalice-automatic detection of authentication bypass vulnerabilities in binary firmware[C]//NDSS. 2015, 1: 1.1-8.1.

[26] GAUR P, TAHILIANI M P. Operating Systems for IoT Devices: A Critical Survey[C/OL]//2015 IEEE Region 10 Symposium. 2015: 33-36.

[27] 姚文详. ARM Cortex-M3 与 Cortex-M4 权威指南[M]. 清华大学出版社, 2015. 354-363.

[28] Quynh N A, Vu D H. Unicorn: Next generation cpu emulator framework[J]. BlackHat USA, 2015, 476.

[29] Qiling Framework[EB/OL]. https://qiling.io/.

[30] STEPHENS N, GROSEN J, SALLS C, et al. Driller: Augmenting fuzzing through selective symbolic execution.[C]//NDSS: 卷 16. 2016: 1-16.

[31] YUN I, LEE S, XU M, et al. {QSYM}: A practical concolic execution engine tailored for hybrid fuzzing[C]//27th {USENIX} Security Symposium ({USENIX} Security 18). 2018: 745-761.

[32] CHEN P, CHEN H. Angora: Efficient fuzzing by principled search[C]//2018 IEEE Symposium on Security and Privacy (SP). IEEE, 2018: 711-725.

[33] ZHENG Y, DAVANIAN A, YIN H, et al. FIRM-AFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation.[C]//USENIX Security Symposium. 2019: 1099-1114.